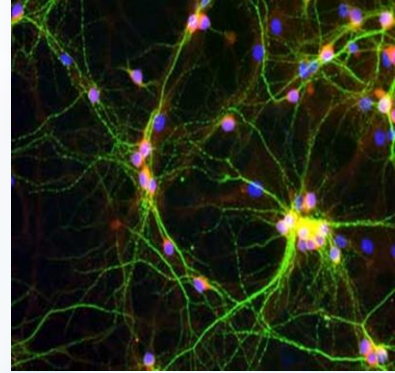


Artificial Neural Networks

Lecture 3



Plutarch: the brain is not a container needs to be filled with knowledge, but a torch needs to be lighted





Google AutoCar

优酷



Neural Arts



Are these pictures created by ANN? If so, how does the ANN make it?

Neural Arts





Part 0: What is ANN?





1.1 ANN

- ❖ **Individual units are interconnected** that form a **graph** with many types of structures - acyclic or cyclic, directed or undirected.

Connectionism

- ❖ **Bottom-Up AI**

Motivated by biological neural systems

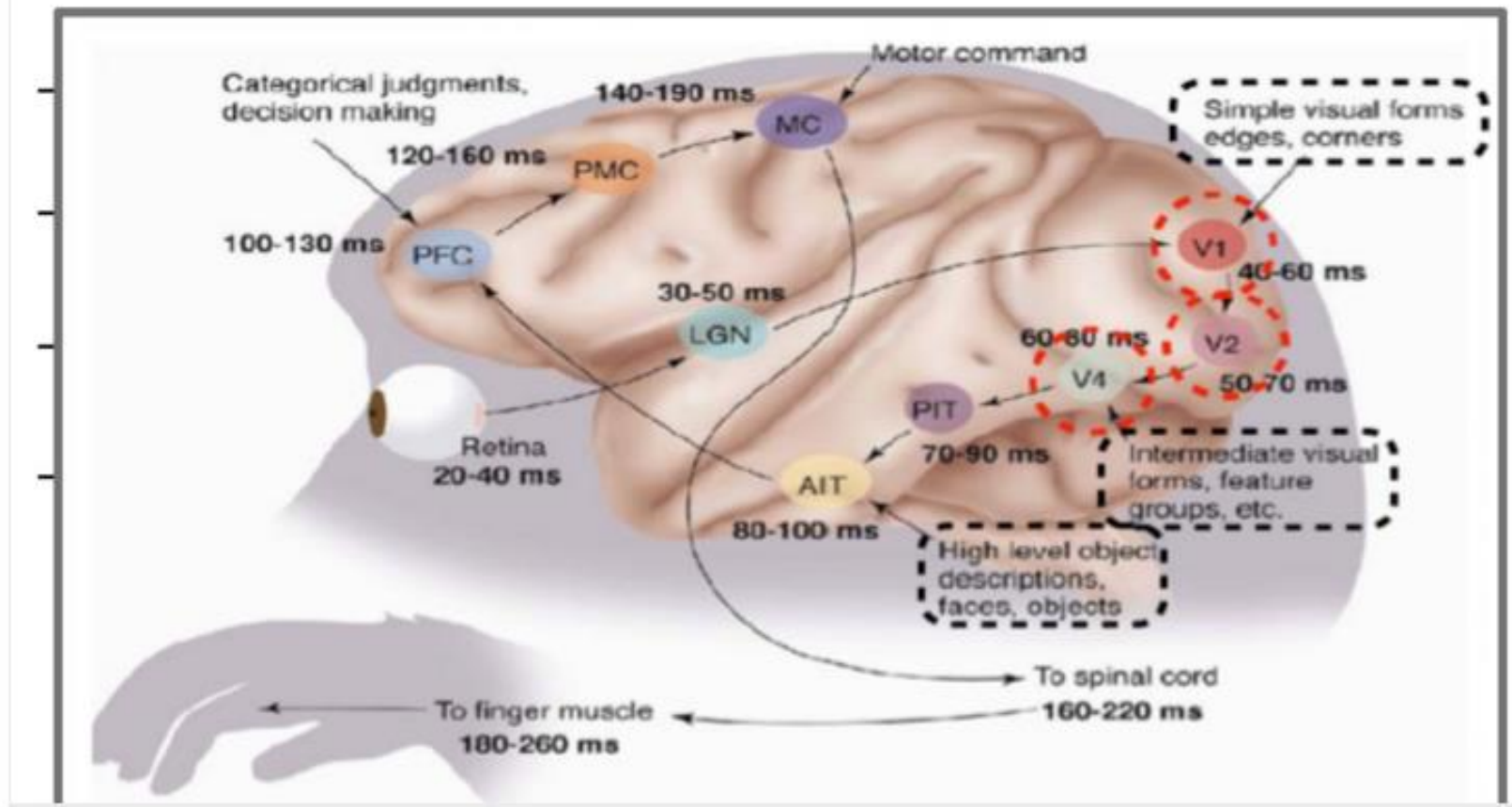
from simulation of structure to simulation of functionality



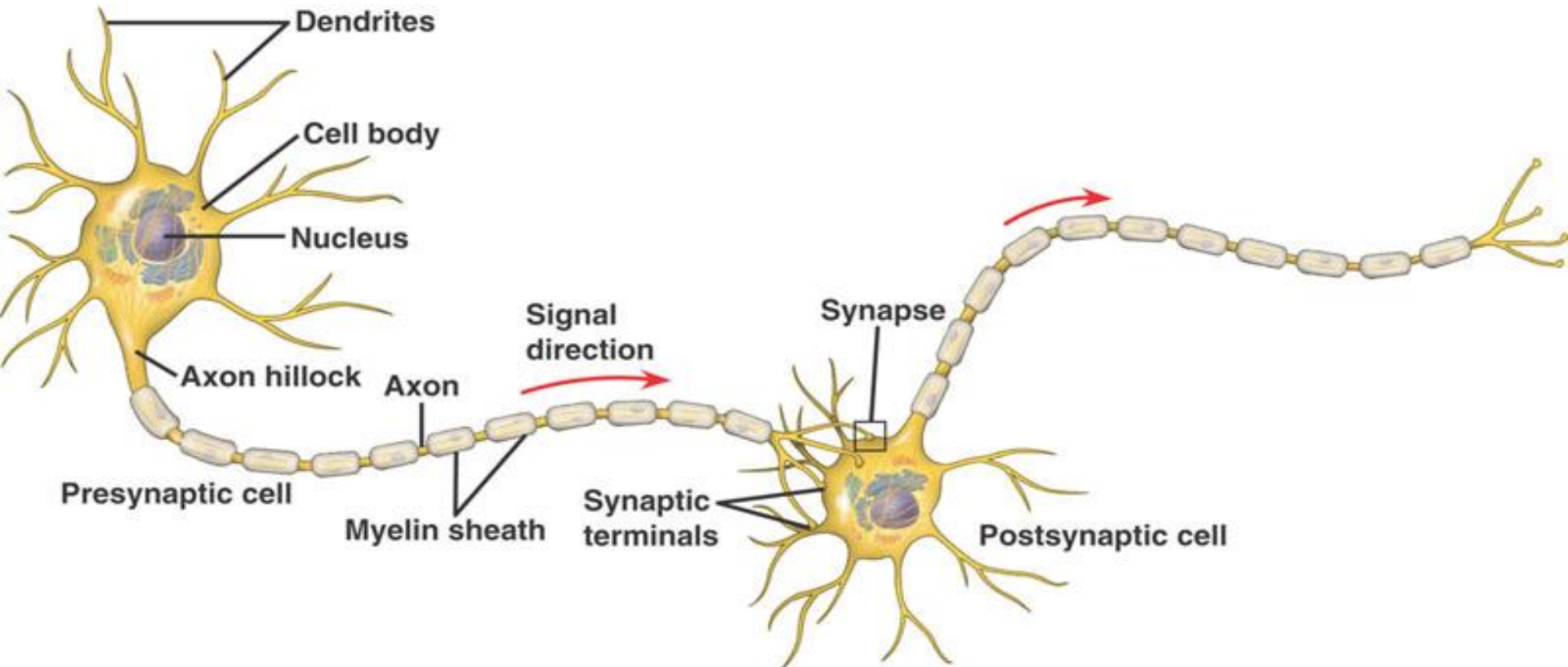
1.1 ANN

Biological Inspiration: Biological Neural System

It is estimated the number of neurons in the human brain (cortex) to be around 10^{10} or 10^{11} .



Biological Neurons

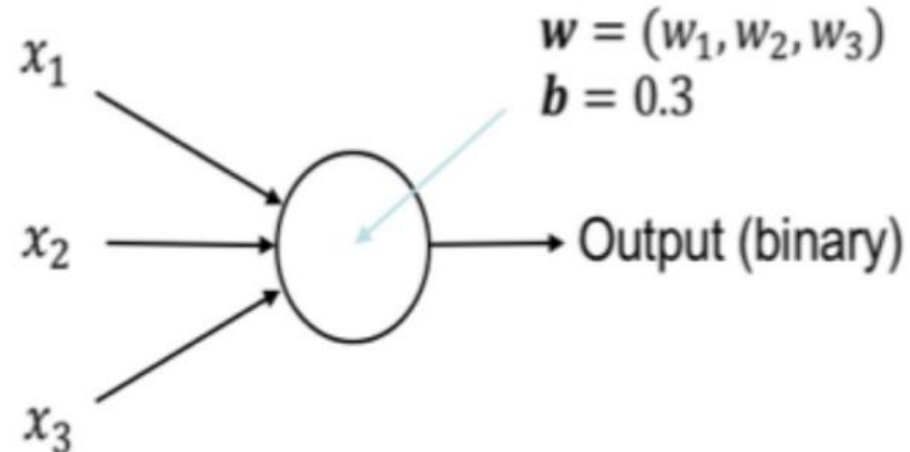
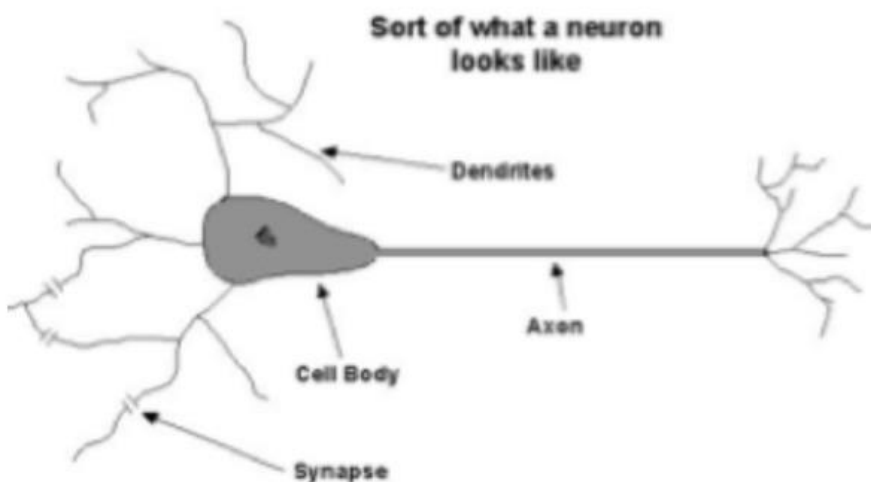


- Temporal and Spatial Summation
- Excitation and Inhibition

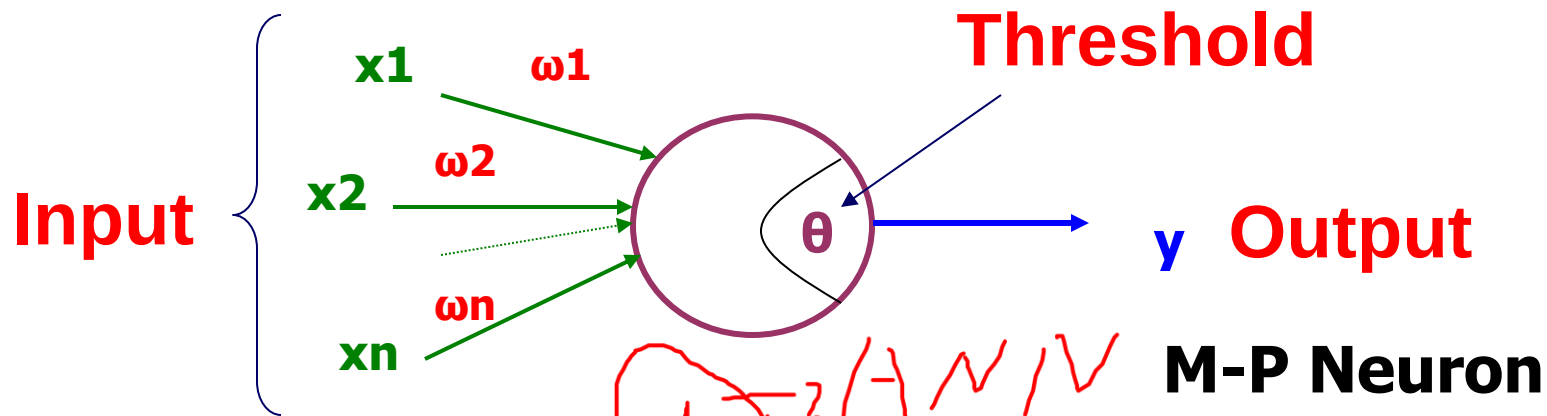
Picture:
http://kvhs.nbed.nb.ca/gallant/biology/neuron_structure.jpg

Inspiration: Biological Neurons

- Neurons
 - accept information from multiple inputs,
 - transmit (process and relay) information to other neurons.
- Multiply inputs by weights along edges
- Apply some **activation function** to the output of each node



Artificial Neurons



$$y = f(\xi) = f(\underline{g(X)})$$

f: Activation Function

g: Combination Function

McCulloch and Pitts, *A logical calculus of the ideas immanent in nervous activity*, 1943



❖ Weighted Sum

$$\xi = g(\mathbf{X}) = \sum_{i=1}^n \omega_i x_i - \theta$$

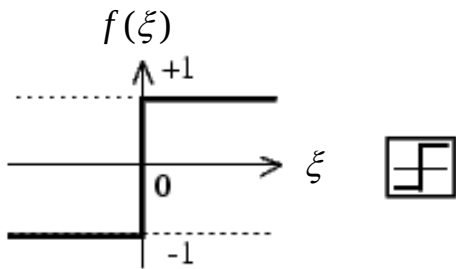
$$y = f(\xi) = f\left(\sum_{i=1}^n \omega_i x_i - \theta\right) \longrightarrow y = f(\xi) = f\left(\sum_{i=0}^n \omega_i x_i\right)$$

❖ Radial Distance

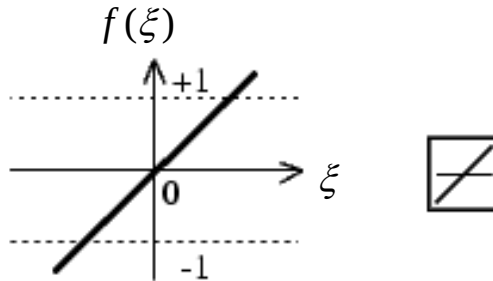
$$\xi = \|\mathbf{X} - \mathbf{C}\| = \sqrt{\sum_{i=1}^n (x_i - c_i)^2} \qquad y = f(\xi) = f\left(\sqrt{\sum_{i=1}^n (x_i - c_i)^2}\right)$$



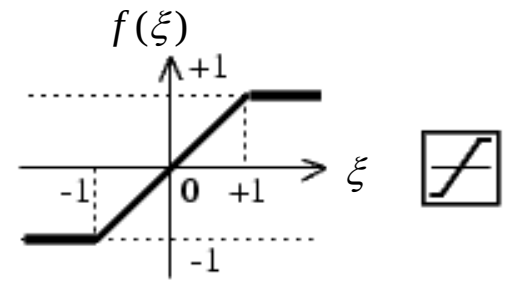
Activation Functions



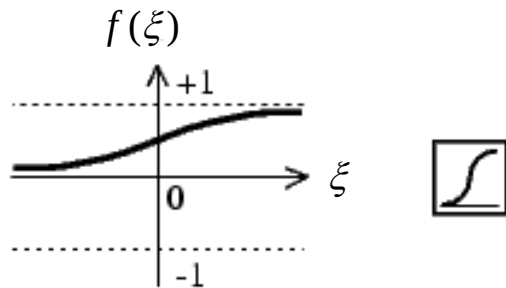
Threshold



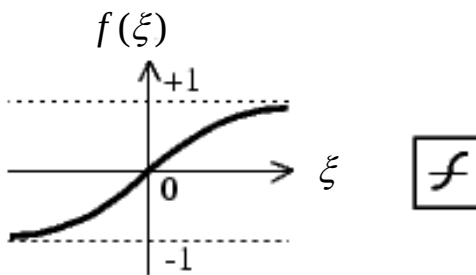
Linear



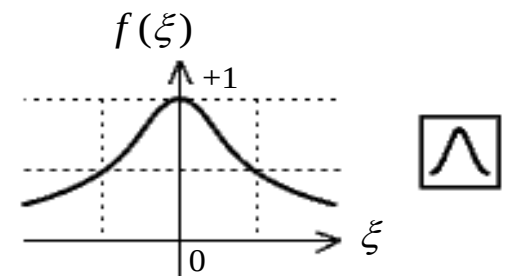
Saturating Linear



Logistic Sigmoid



**Hyperbolic
tangent Sigmoid**



Gaussian

Activation Functions

Other types of activation function

Activation Functions

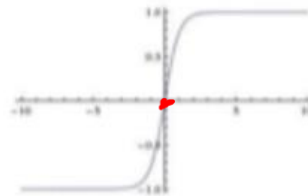
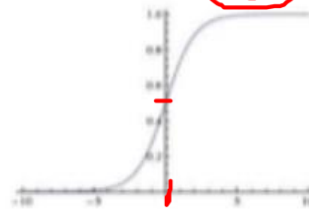
$$\delta(x) = 1 - \delta(-x)$$

Sigmoid

$$\sigma(x) = 1 / (1 + e^{-x})$$

$$\tanh \quad \tanh(x)$$

$$\text{ReLU} \quad \max(0, x)$$



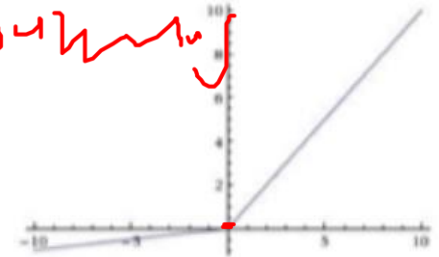
① vanishing gradient

② non-0

③ time consuming

Leaky ReLU

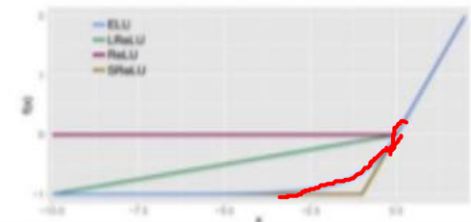
$$\max(0.1x, x)$$



$$\text{Maxout} \quad \max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha (\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$



① dead

② non-zero

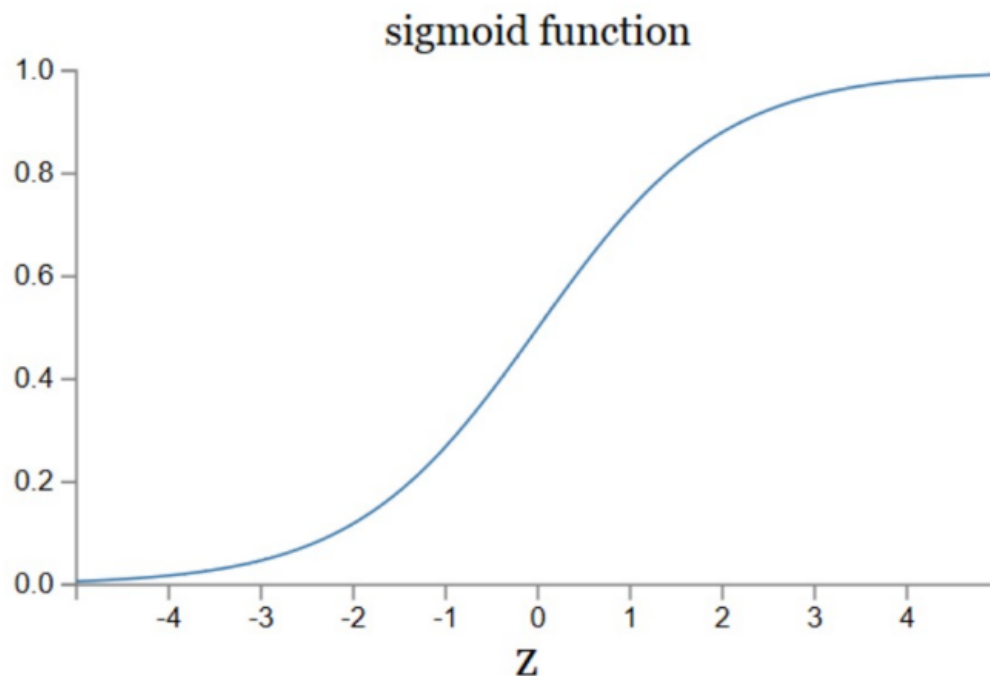
Activation Functions

Sigmoid non-linear activation

- We're going to introduce non-linear activation function to transform the features.

$$\sigma(w \cdot x + b)$$

$$\sigma(z) \equiv \frac{1}{1 + e^{-z}}$$

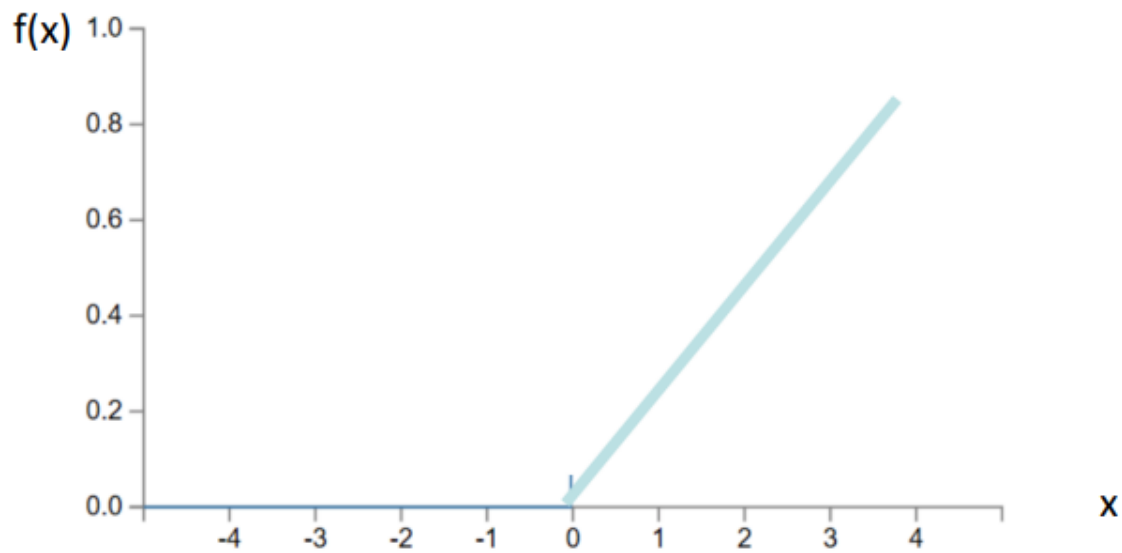


Activation Functions

Rectified Linear Unit (ReLU)

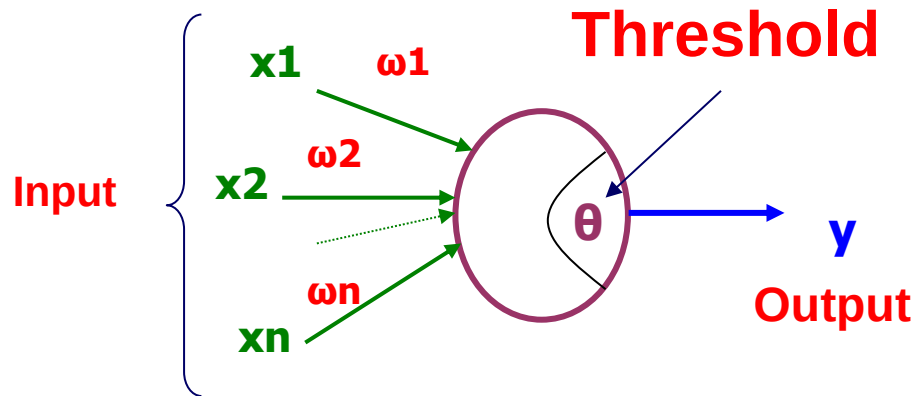
- ReLU

$$f(x) = \max(0, x)$$



Artificial Neurons

M-P Neuron



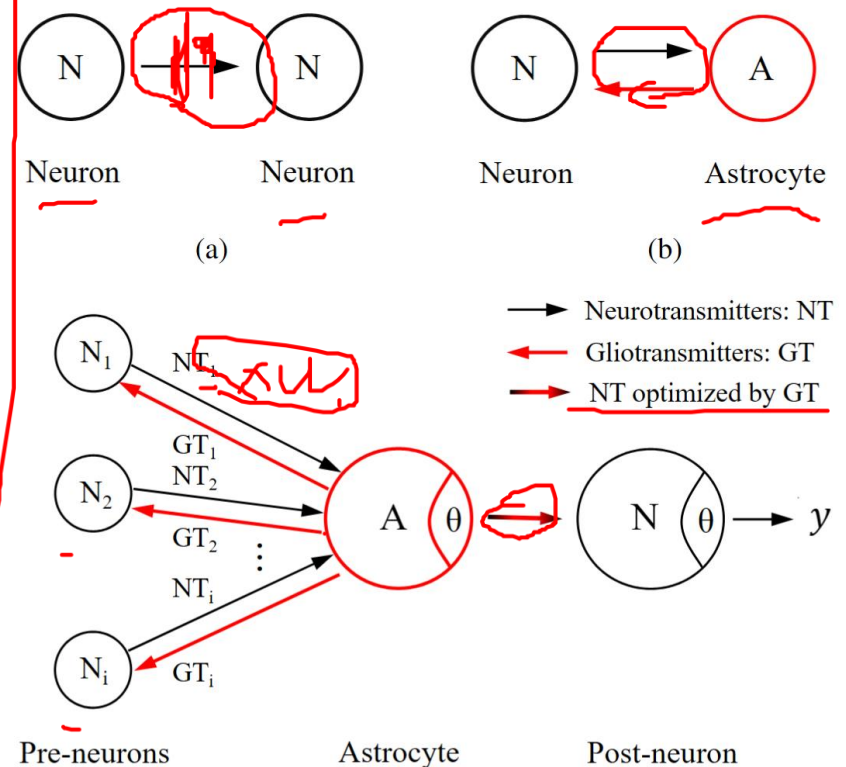
x learned

$$T = w \cdot x$$

$$Y = f(x)$$

Asternet

Glia

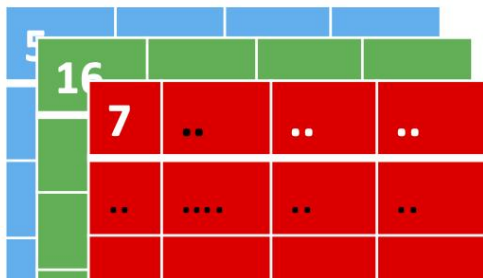


Binary Classification Example

- Is it a Kangaroo?



$\{0,1\}$
 1 – kangaroo
 0 – not a kangaroo



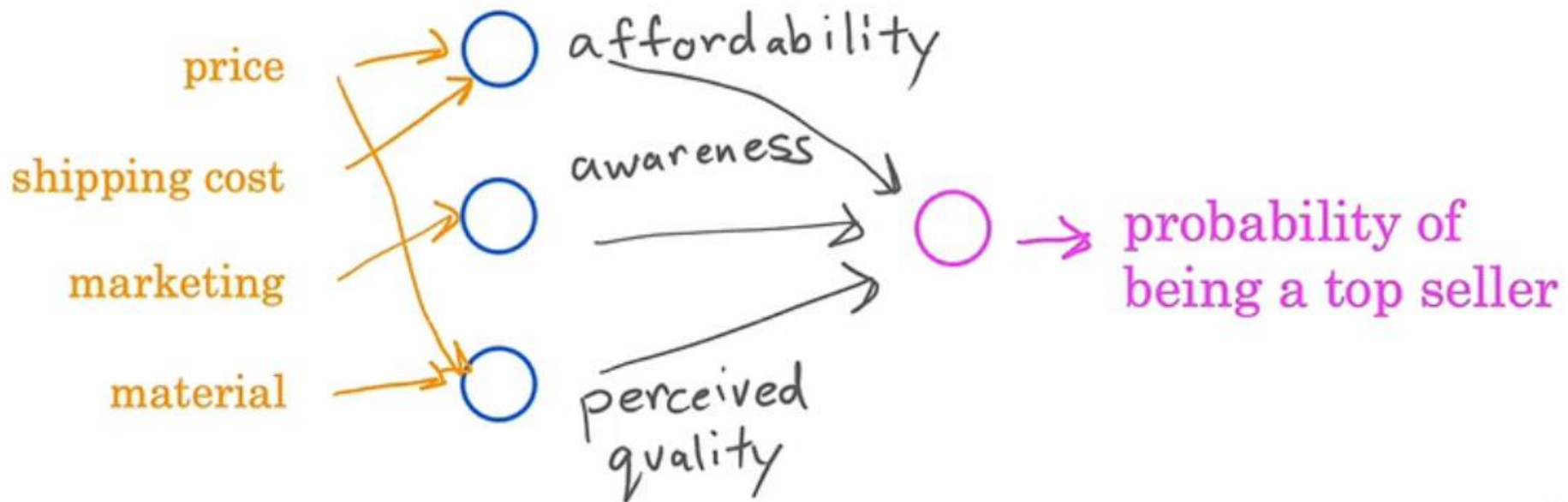
$$X = \begin{bmatrix} 5 \\ \ddots \\ \ddots \\ 16 \\ \ddots \\ \ddots \\ 7 \\ \ddots \end{bmatrix}$$

X Dim is $n_x \times m \times 3$
 N_x is num features

Y dim is 1
 $X \rightarrow Y$

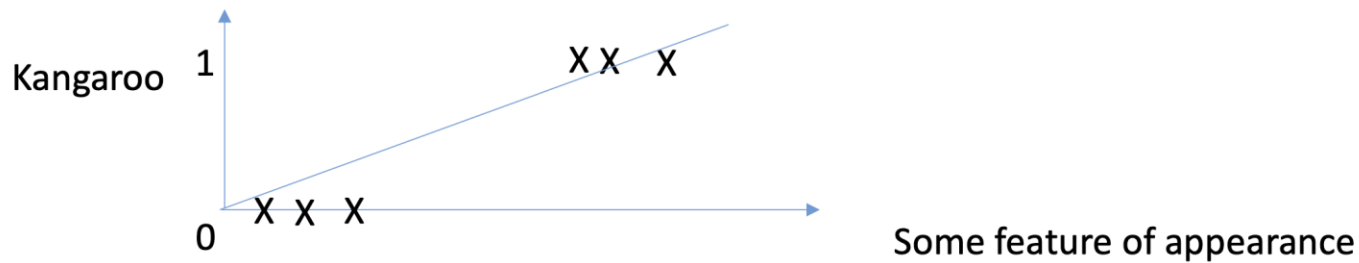
Credit: Nick Barnes

Binary Classification Example



Linear Regression Vs Logistic Regression

- In 1 dimension

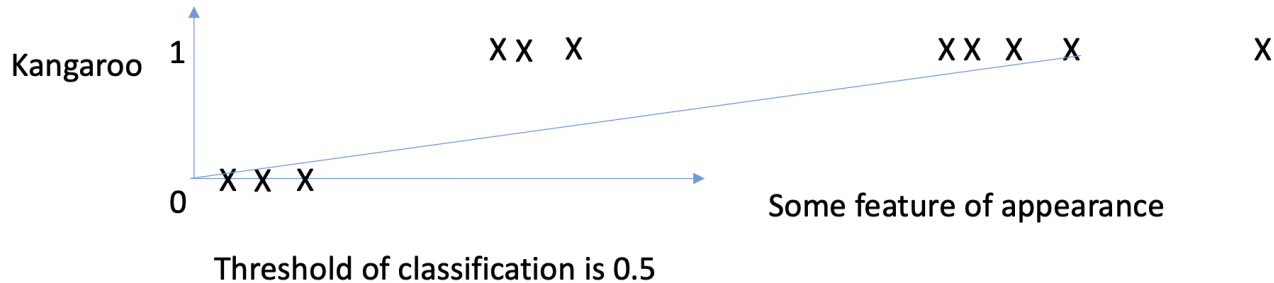


Threshold of classification is 0.5

Credits: Nick Barnes

Linear Regression Vs Logistic Regression

- But what about



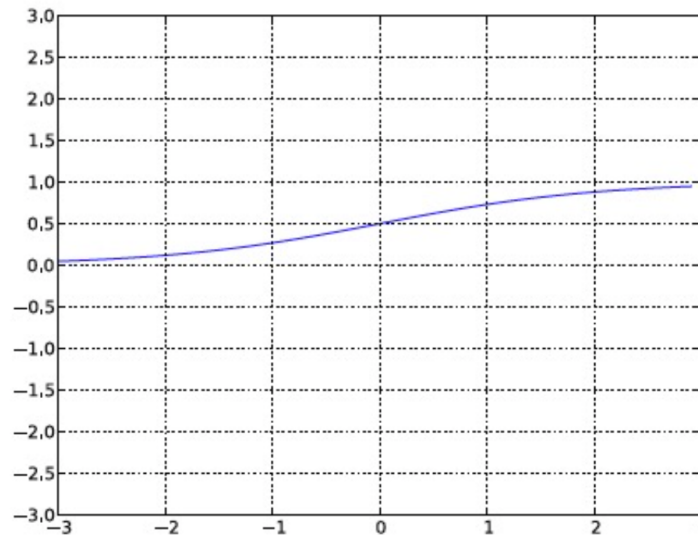
- Its not really what we want
- The decision boundary is being dominated by distant points.
- We want a function that is mostly zero below 0.5 and mostly 1 above 0.5.

Credits: Nick Barnes

Logistic Regression

- Sigmoid function
 - Squashes the curve between 0 and 1
 - Always positive
 - Bounded
 - Strictly increasing
 - Continuous
 - Differentiable

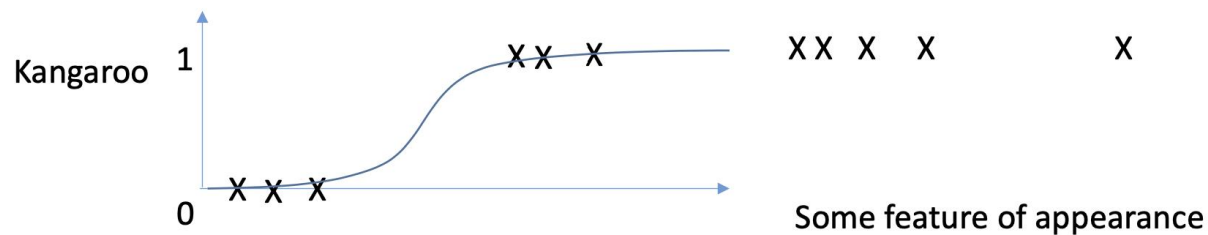
$$g(a) = \text{sigm}(a) = \frac{1}{1 + \exp(-a)}$$



Credits: Nick Barnes

Logistic Regression

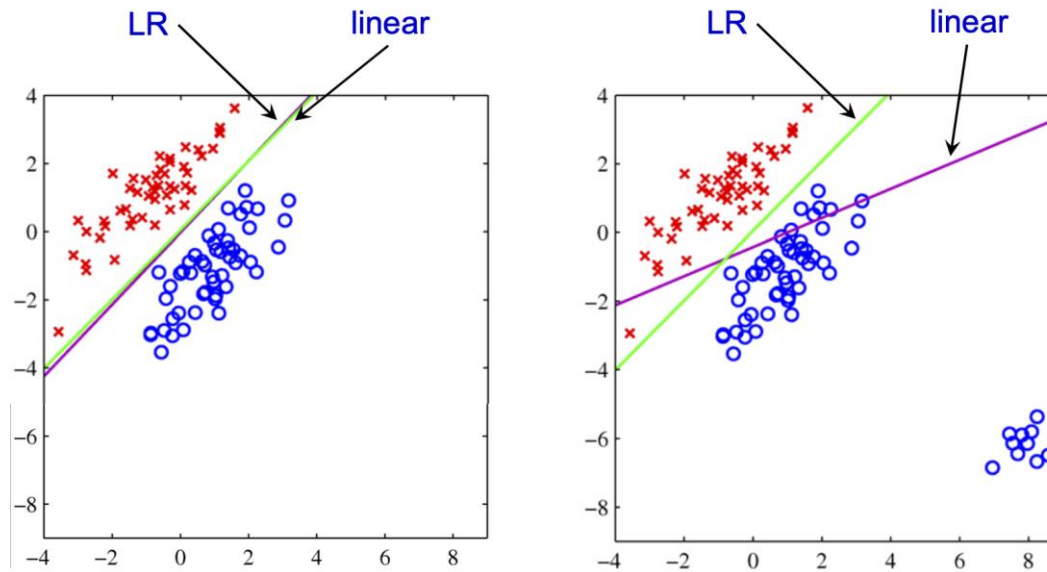
❖ Predict $y = 1$ if $f(x) \geq 0.5$,
otherwise predict $y=0$ if $f(x) < 0.5$



$$g(a) = \text{sigm}(a) = \frac{1}{1 + \exp(-a)}$$

Model is a sigmoid function (or a logistic function)

Logistic Regression



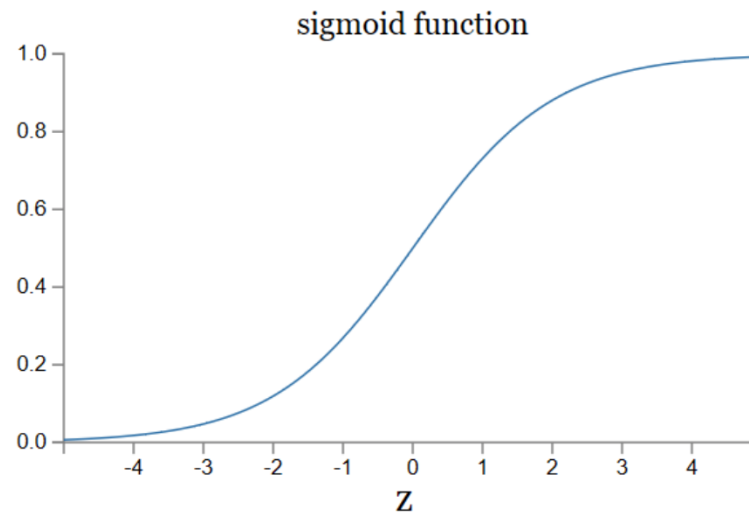
$\sigma(w_1x_1 + w_2x_2 + b)$ fit, vs $w_1x_1 + w_2x_2 + b$

Credit: A. Zisserman

Sigmoid non-linear activation

Sigmoid, non-linear function, is used to transform features.

$$\sigma(w \cdot x + b)$$
$$\sigma(z) \equiv \frac{1}{1 + e^{-z}}.$$



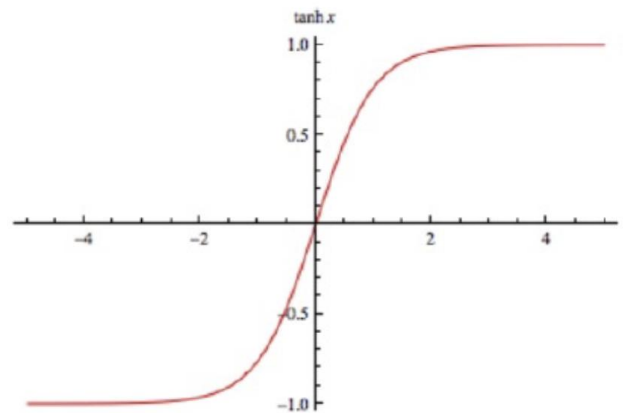
Nielsen

Activation Functions

1. Sigmoid function.
2. Hyperbolic tangent.

- Squash between -1 and 1
- Positive or negative
- Bounded
- Strictly increasing

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$



Credit: Nick Barnes

Activation Functions

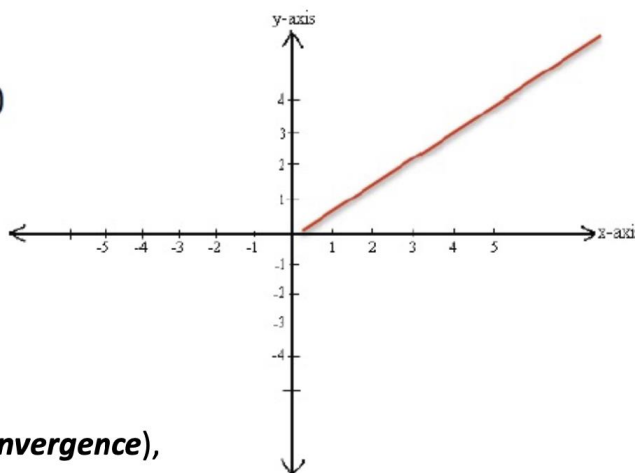
1. Sigmoid function.
2. Hyperbolic tangent.
3. Linear Rectifier (ReLU)
 - Bounded below by 0
 - Not upper bounded
 - Strictly increasing
 - Sparse neurons

Does not saturate (>0)

Very computationally efficient (no exp)

Converges much faster (*e.g. 6x faster convergence*),

$$\text{rectifier}(x) = \max(0, x)$$

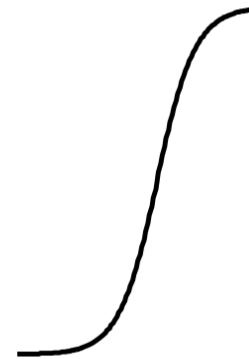
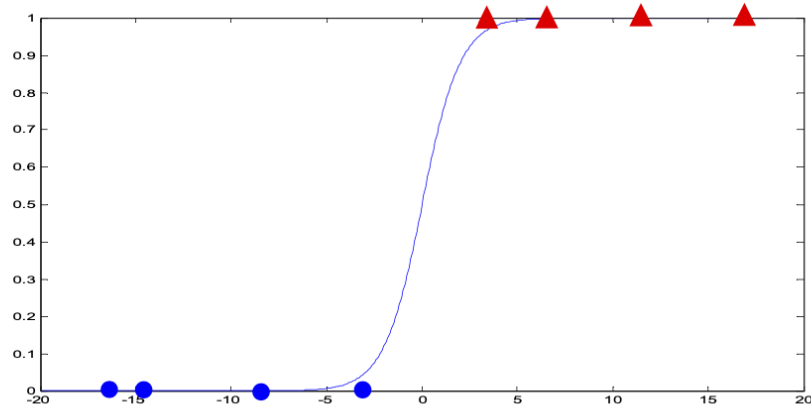


AlexNet [Krizhevsky et al., 2012]

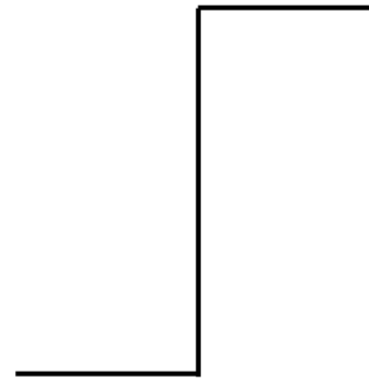
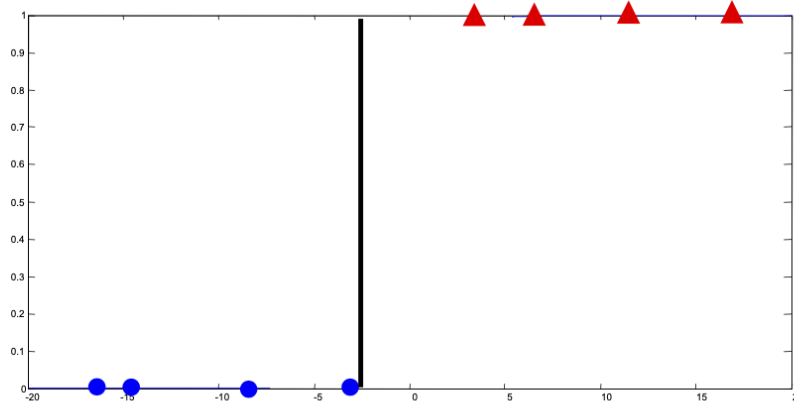
Used it.

Glorot, 2011 introduced to deep nets

Deep Sparse Rectifier Neural Network



- A sigmoid favours a larger margin compared with a step classifier.

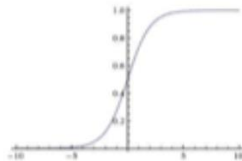


Types of activation function

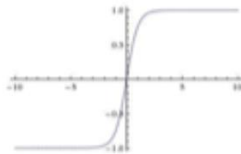
Activation Functions

Sigmoid

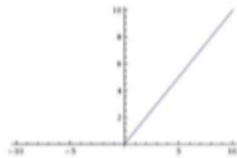
$$\sigma(x) = 1/(1 + e^{-x})$$



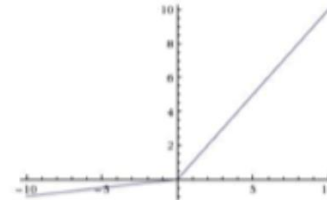
tanh tanh(x)



ReLU max(0,x)

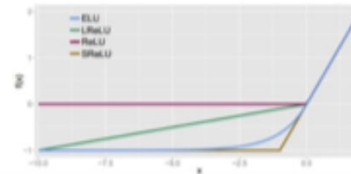


Leaky ReLU max(0.1x, x)



Maxout $\max(w_1^T x + b_1, w_2^T x + b_2)$

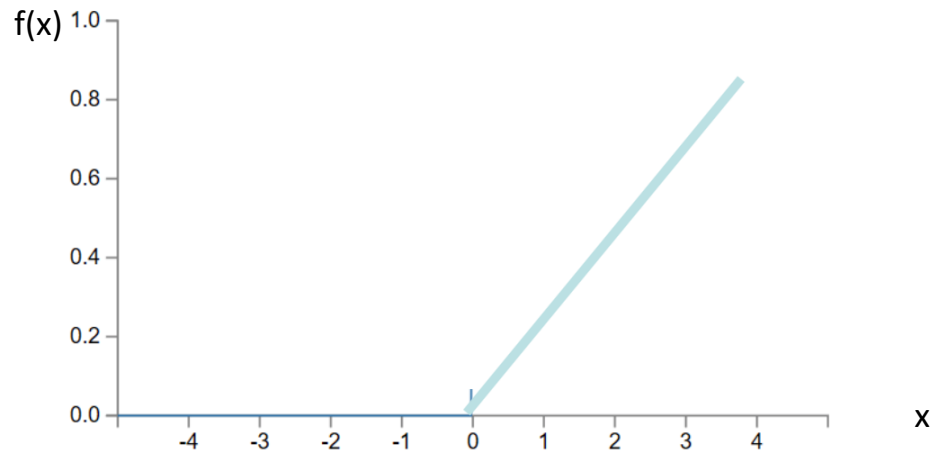
$$\text{ELU} \quad f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha (\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$



Another non-linear activation function: Rectified Linear Unit (ReLU)

- ReLU

$$f(x) = \max(0, x)$$



Artificial Neurons

Binary Classifying an image

- Each pixel of the image would be an input.
- So, for a 28 x 28 image, we vectorize.
- $\mathbf{x} = 1 \times 784$
- $\mathbf{w}$ is a vector of weights for each pixel, 784×1
- b is a scalar bias per perceptron
- $\text{result} = \mathbf{xw} + b \quad \rightarrow (1 \times 784) \times (784 \times 1) + b = (1 \times 1) + b$

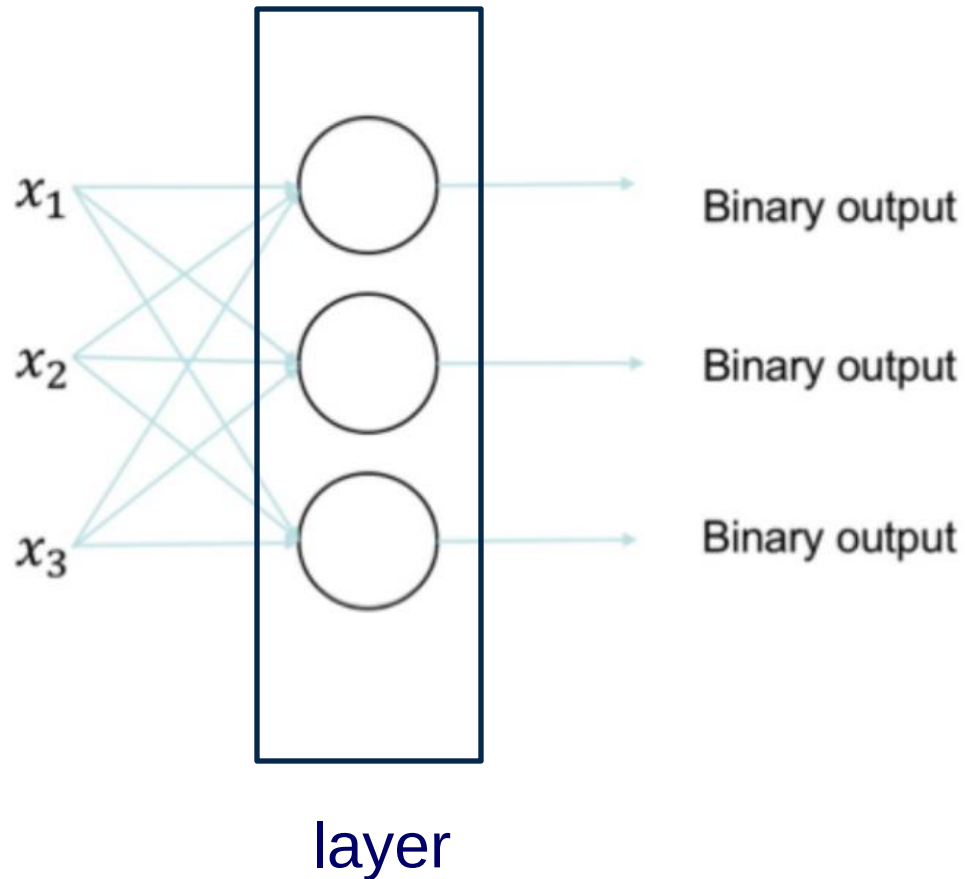


Multi-class classification

- Each pixel of the image would be an input node.
- So, for a 28x28 image, we vectorize the input image as a $\mathbf{x}=1 \times 784$ for $\#(\text{classes}) = \text{e.g., } 10$, in MNIST digit recognition
- $\mathbf{W}$ is a matrix of weights for each pixel/each perceptron – $\mathbf{W} = 784 \times 10$ (10-class classification)
- $\mathbf{b}$ is a bias *per perceptron* (vector of biases); (1×10)
- $\text{result} = \mathbf{x}\mathbf{W} + \mathbf{b} \rightarrow (1 \times 784) \times (784 \times 10) + \mathbf{b} \rightarrow \underline{(1 \times 10)} + (1 \times 10) = \underline{10\text{-D}}$
output vector

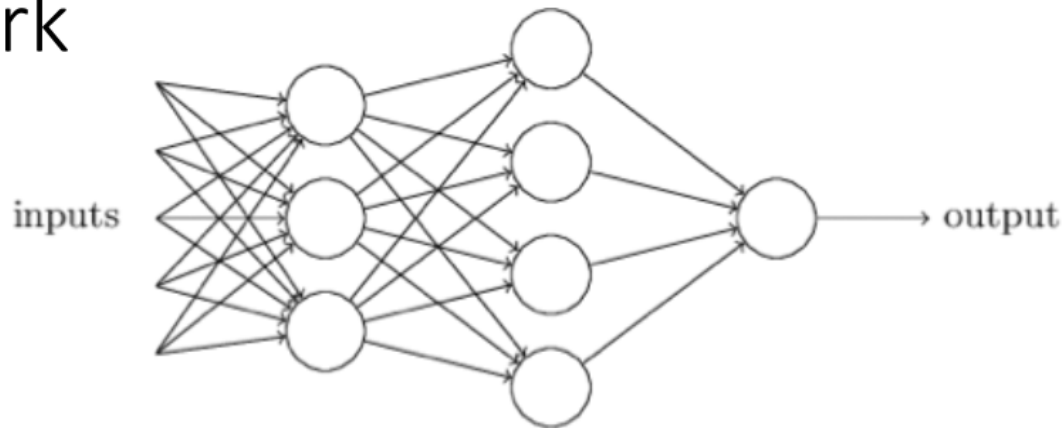
Artificial Neurons

Add more neurons -> Multiclass neural networks



Multiple Layer neural network

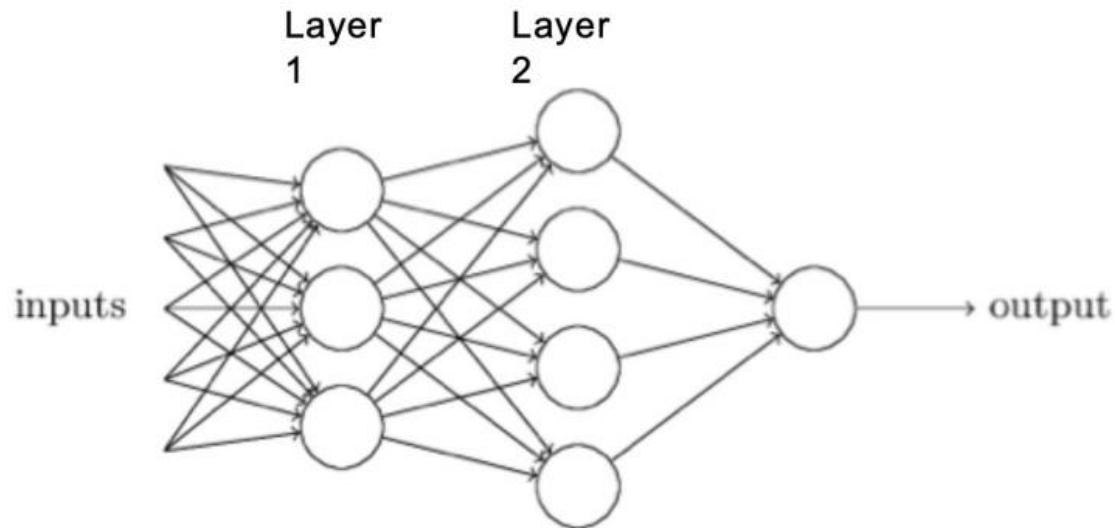
Add more layers -> Multiple Layer neural network



- Attempt to represent complex functions as compositions of smaller functions.
- Outputs from one perceptron are fed into inputs of another perceptron.

Multiple Layer neural network

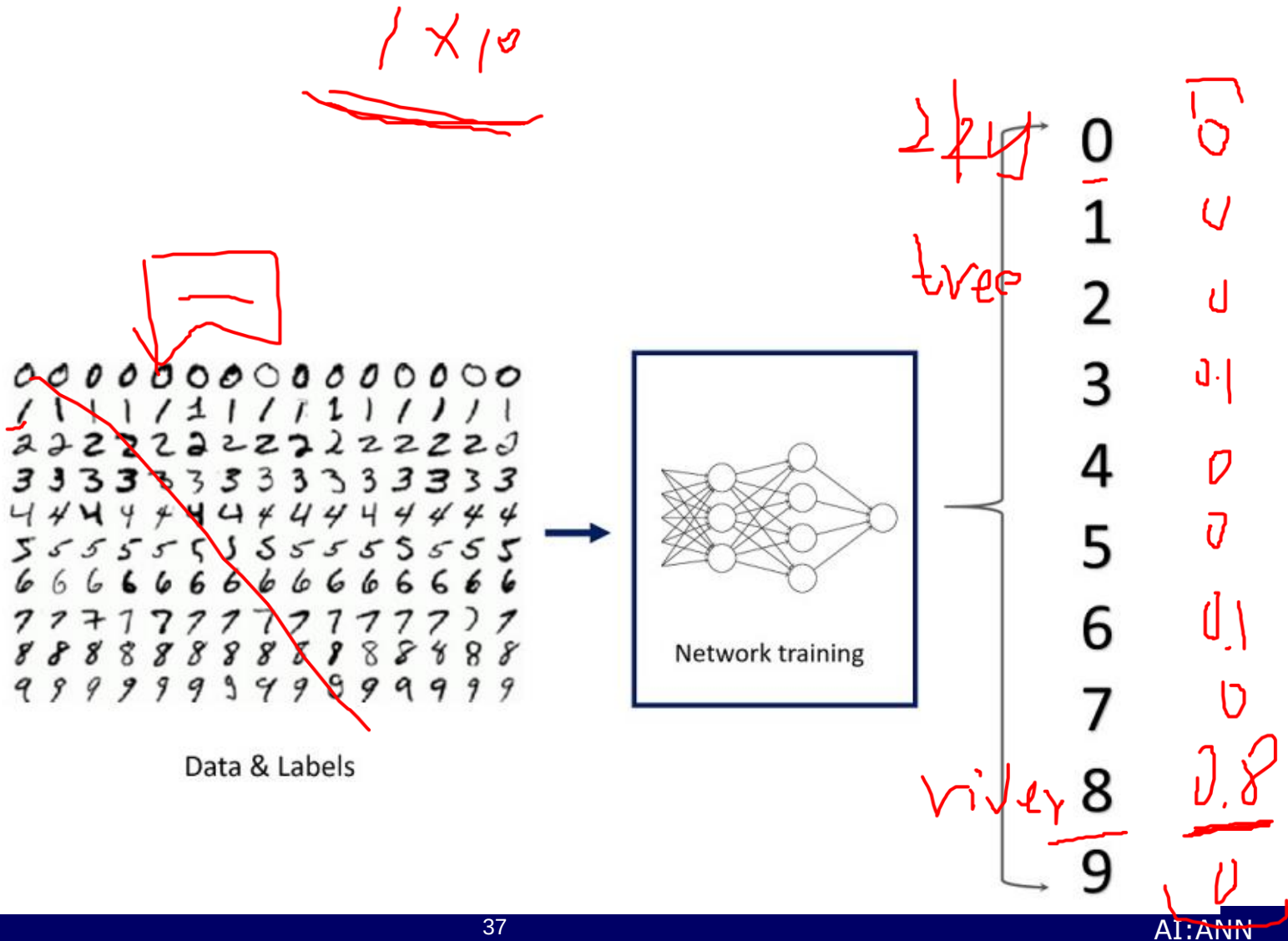
Composition



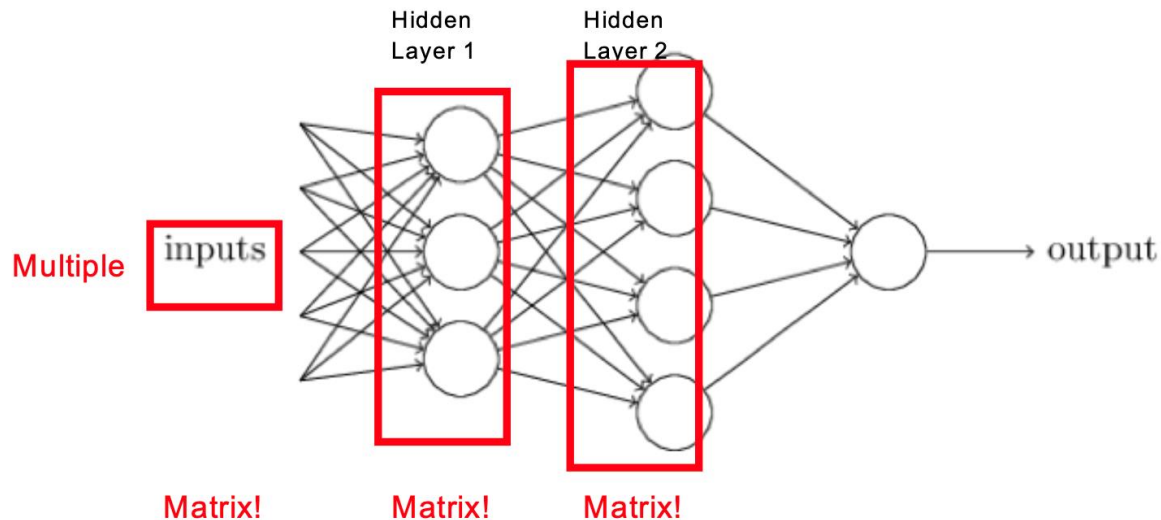
Sets of layers and the connections (weights) between them define the *network architecture*.

Multiple Layer neural network

Application: Multi-class classification



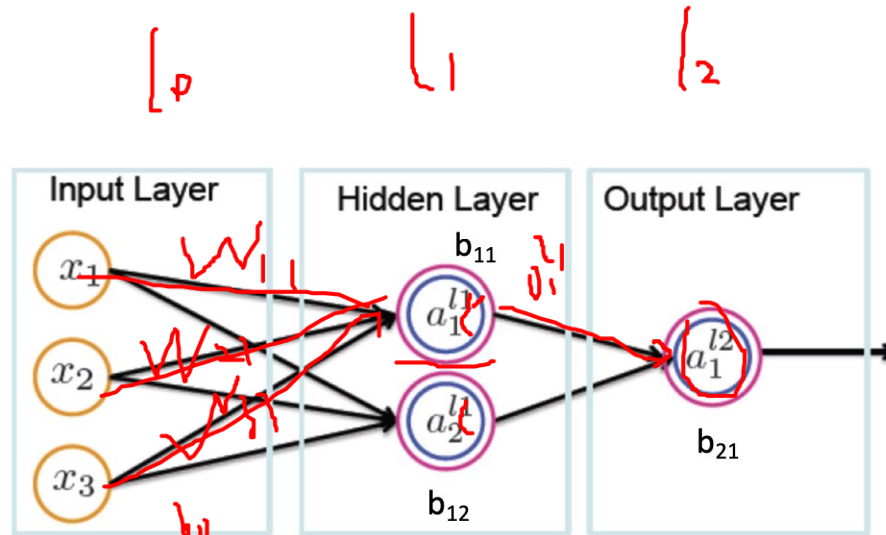
Parallel computation



It's all just matrix-vector multiplications !
GPUs -> special hardware for fast/large matrix multiplication.

Nielsen

Single Hidden Layer Example



$$a_1^{l1} = w_{11}^{l0} x_1 + w_{21}^{l0} x_2 + w_{31}^{l0} x_3 + b_{11}$$

$$o_1^{l1} = g(a_1^{l1})$$

$$o_1^{l1} = g(w_{11}^{l0} x_1 + w_{21}^{l0} x_2 + w_{31}^{l0} x_3 + b_{11}) \quad \rightarrow \quad o_1^{l2} = g(w_{11}^{l1} o_1^{l1} + w_{12}^{l1} o_2^{l1} + b_{21})$$

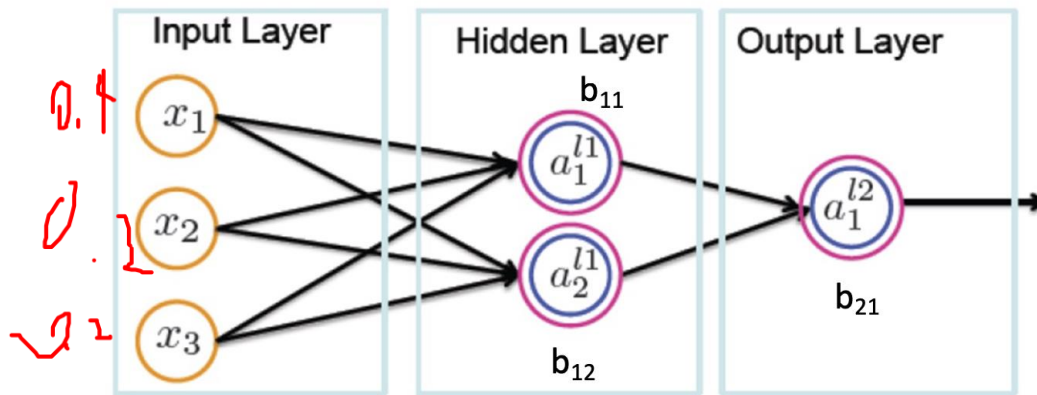
$$o_2^{l1} = g(w_{12}^{l0} x_1 + w_{22}^{l0} x_2 + w_{32}^{l0} x_3 + b_{12})$$

Output Layer: $o_1^{l2} = g(w^{l1} g(w^{l0} x))$

L1 -> Layer 1
1: 1-st neuron

21: 2-nd weight on the 1-st neuron

Single Hidden Layer Example



Example, suppose:

$X = \{0.4, 0.2, 0.2\}$

$w_{1.}^{l0} = \{1.0, 0.5\}$

$w_{2.}^{l0} = \{0.2, 1.2\}$

$w_{3.}^{l0} = \{1.2, 0.1\}$

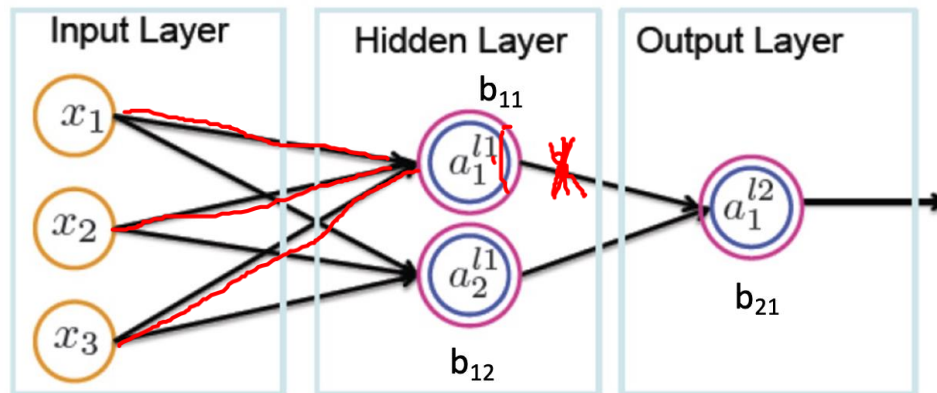
$w_{1.}^{l1} = \{0.3, 0.9\}$

$B_{11} = 0.1$

$B_{12} = 0.2$

$B_{21} = 0.3$

Compute!



Example, suppose:

$$X = \{0.4, 0.2, 0.2\}$$

$$w_{1.}^{l0} = \{1.0, 0.5\}$$

$$w_{2.}^{l0} = \{0.2, 1.2\}$$

$$w_{3.}^{l0} = \{1.2, 0.1\}$$

$$w_{1.}^{l1} = \{0.3, 0.9\}$$

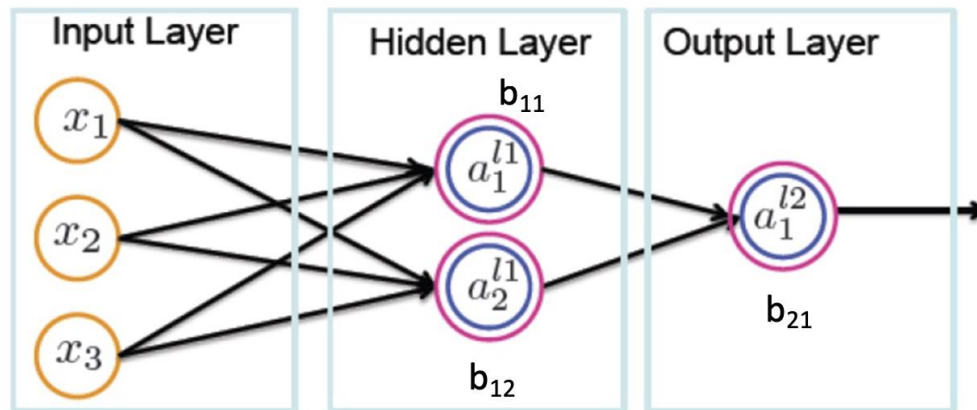
$$B_{11} = 0.1$$

$$B_{12} = 0.2$$

$$B_{21} = 0.3$$

$$\begin{aligned} o_1^{l1} &= g(w_{11}^{l0} x_1 + w_{21}^{l0} x_2 + w_{31}^{l0} x_3 + b_{11}) \\ &= g(1.0 * 0.4 + 0.2 * 0.2 + 1.2 * 0.2 - 0.1) \end{aligned}$$

Single Hidden Layer Example



Example, suppose:

$$X = \{0.4, 0.2, 0.2\}$$

$$w_{1.}^{l0} = \{1.0, 0.5\}$$

$$w_{2.}^{l0} = \{0.2, 1.2\}$$

$$w_{3.}^{l0} = \{1.2, 0.1\}$$

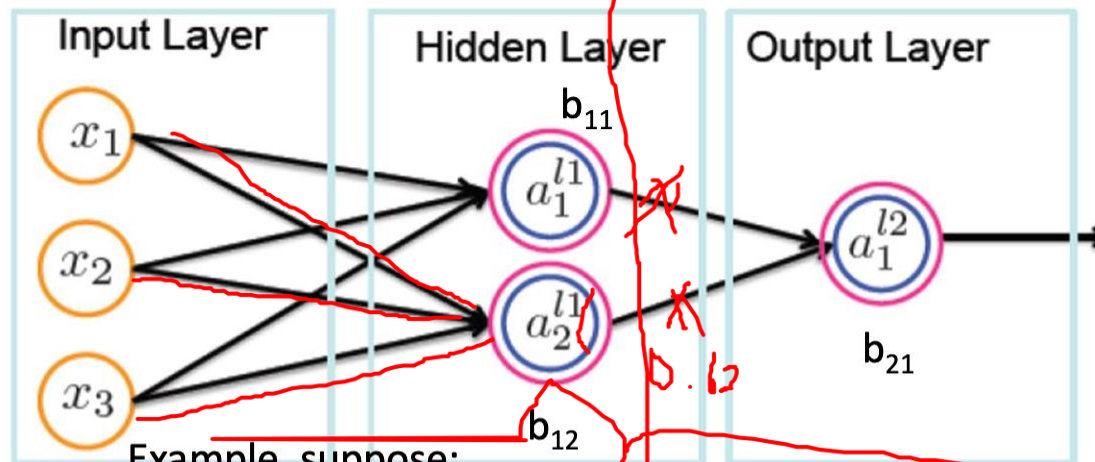
$$w_{1.}^{l1} = \{0.3, 0.9\}$$

$$B_{11} = 0.1$$

$$B_{12} = 0.2$$

$$B_{21} = 0.3$$

$$\begin{aligned} o_1^{l1} &= g(w_{11}^{l0} x_1 + w_{21}^{l0} x_2 + w_{31}^{l0} x_3 + b_{11}) \\ &= g(1.0 * 0.4 + 0.2 * 0.2 + 1.2 * 0.2 + 0.1) \\ &= g(0.4 + 0.04 + 0.24 + 0.1) \\ &= g(0.78) \\ &= 1/(1 + \exp(-0.78)) \end{aligned}$$



Example, suppose:

$$X = \{0.4, 0.2, 0.2\}$$

$$w_{1.}^{l0} = \{1.0, 0.5\}$$

$$w_{2.}^{l0} = \{0.2, 1.2\}$$

$$w_{3.}^{l0} = \{1.2, 0.1\}$$

$$w_{1.}^{l1} = \{0.3, 0.9\}$$

$$B_{11} = 0.1$$

$$B_{12} = 0.2$$

$$B_{21} = 0.3$$

$$o_2^{l1} = g(w_{12}^{l0} x_1 + w_{22}^{l0} x_2 + w_{32}^{l0} x_3 + b_{12})$$

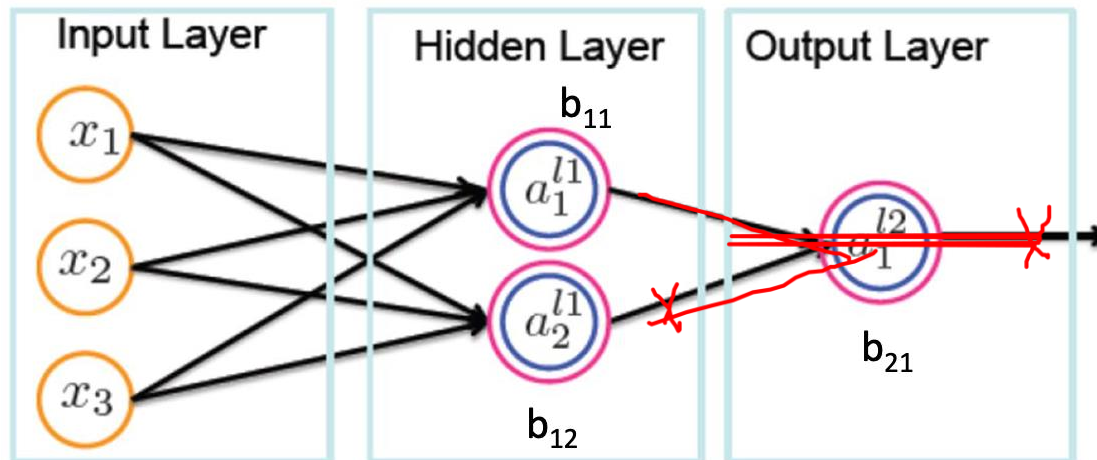
$$= g(0.5 * 0.4 + 1.2 * 0.2 + 0.1 * 0.2 + 0.2)$$

$$= g(0.2 + 0.24 + 0.03 + 0.2)$$

$$= g(0.49)$$

$$= 1 / (1 + \exp(-0.49))$$

$$= 0.62..$$

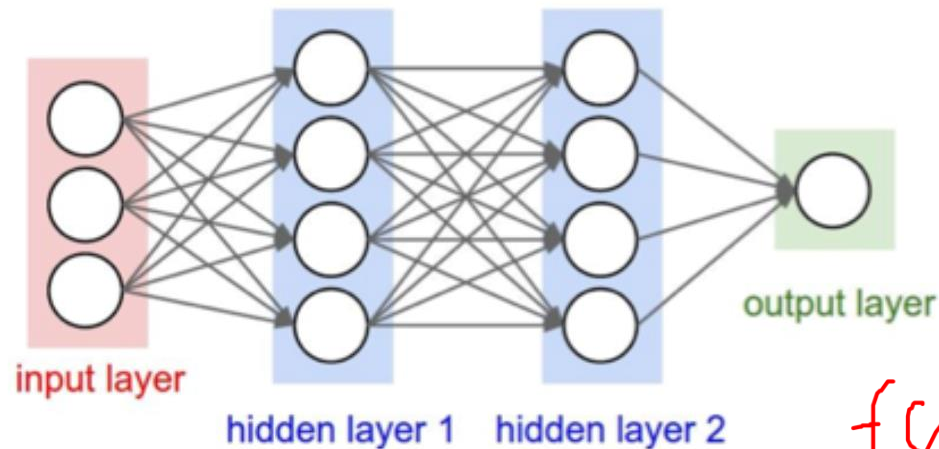


Example, suppose:

$$\begin{aligned}
 X &= \{0.4, 0.2, 0.2\} & o_1^{l2} &= g(w_{11}^{l1} o_1^{l1} + w_{12}^{l1} o_2^{l1} + b_{21}) \\
 w_{1.}^{l1} &= \{1.0, 0.5\} & &= g(0.3 * 0.68 + 0.9 * 0.62 + 0.3) \\
 w_{2.}^{l1} &= \{0.2, 1.2\} & &= g(0.204 + 0.558 + 0.3) \\
 w_{3.}^{l1} &= \{1.2, 0.1\} & &= g(1.062) \\
 w_{1.}^{l2} &= \{0.3, 0.9\} & &= 1 / (1 + \exp(-1.062)) \\
 B_{11} &= 0.1 & &= 0.74.. \\
 B_{12} &= 0.2 & & \\
 B_{21} &= 0.3 & &
 \end{aligned}$$

Note that the computation of the activations at layer 1 are vector computations, and so can be parallelized in a GPU!
Many computations are like this so GPU's greatly decrease computation time (but not order).

Parallel Computation



$$f(x) = \frac{1}{1 + e^{-x}}$$

```
# forward-pass of a 3-layer neural network:  
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)  
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)  
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)  
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)  
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

以下实例使用 lambda 创建匿名函数，设置一个函数参数 a，函数计算参数 a 加 10，并返回结果：

实例

```
x = lambda a : a + 10  
print(x(5))
```

输出结果为：

15

lambda 函数也可以设置多个参数，参数使用逗号，隔开：

以下实例使用 lambda 创建匿名函数，函数参数 a 与 b 相乘，并返回结果：

实例

```
x = lambda a, b : a * b  
print(x(5, 6))
```

输出结果为：

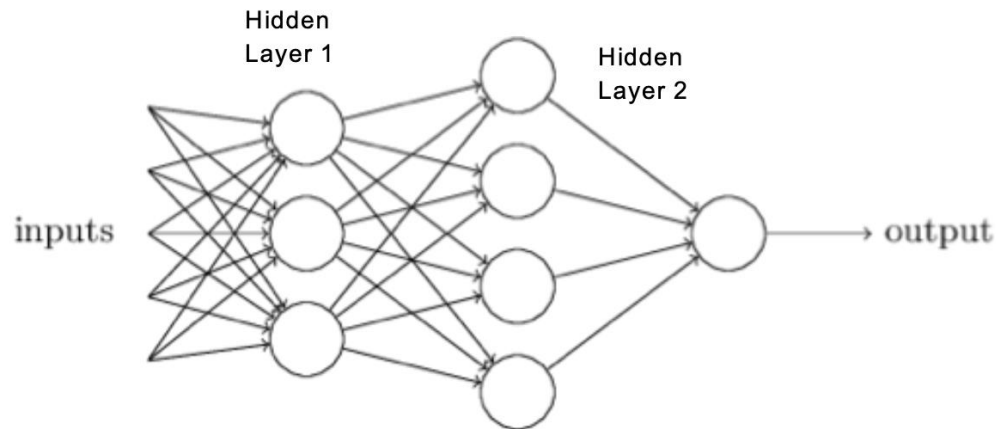
30

Problems with using all-linear functions in the hidden layers

- ❖ **We have formed multiple layers or chains of linear functions.**
- ❖ **We know that linear function of linear functions is a linear function, i.e. they can be reduced:**
 - $g = f(h(x))$
 - Eg. $O_1 = xw_1 + b_1$; $O_2 = o_1w_2 + b_2$; $\Rightarrow O_2 = (xw_1 + b_1)w_2 + b_2 = xw_1w_2 + (b_1w_2 + b_2)$

Our multi-layer composition of functions is really just a single linear function!!!!

Multi-Layer Perceptron (MLP)

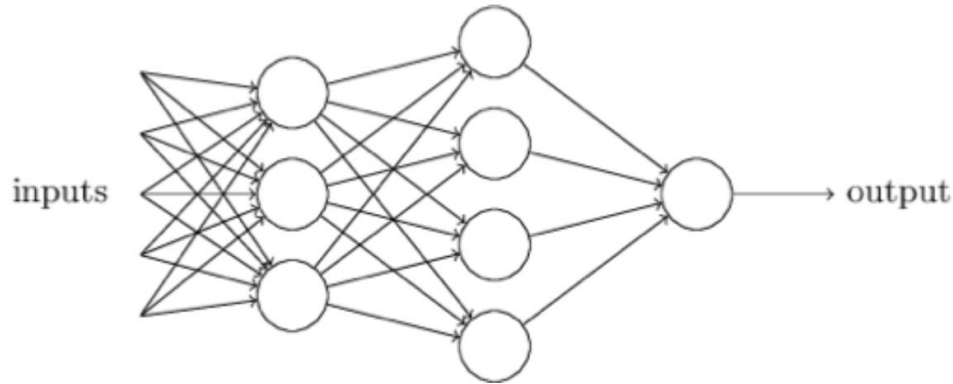


- Attempt to represent complex functions as compositions of smaller functions.
- Outputs from one perceptron are fed into inputs of another perceptron.
- Layers that are in between the input and the output are called *hidden layers*, because we are going to *learn* their weights via an optimization process.

Nielsen

Multi-Layer Perceptron (MLP)

- ...is a '*fully connected*' neural network with non-linear activation functions.



- '*Feed-forward*' neural network

Nielson



Multiple Layer neural network

Mathematical Theorem for MLP

- The application of MLP is grounded in theory.
 - **Universal Approximation Theorem.**

With three (or more) layers and enough parameters, MLP can approximate any function.

Proof(Michael Nielson):

<http://neuralnetworksanddeeplearning.com/chap4.html>

COMP8536: Advanced Topics in Deep Learning for Computer Vision





Multiple Layer neural network

If a 3-layer network can approximate any function ...given enough parameters ... then why should we need to go deeper ?

- Intuitively, composition is efficient because it allows *reuse*.
- Empirically, deeper networks do a better job than shallow networks at learning certain hierarchies of knowledge.





Multiple Layer neural network

Why go deeper ?

- A three-layer network is already a “universal approximator”,
- Only if the number of neurons in the hidden layer is sufficiently many.
- The identification of the weights is troublesome (possible convergence into wrong local minima)
- Deep neural networks
 - Have many levels of non-linearity
 - Compactly represent highly non-linear functions
 - Layered structure facilitates convergence to a good optimum
 - **There are many local optima of approximately the same (and good) quality**



Multiple Layer neural network Interpretation

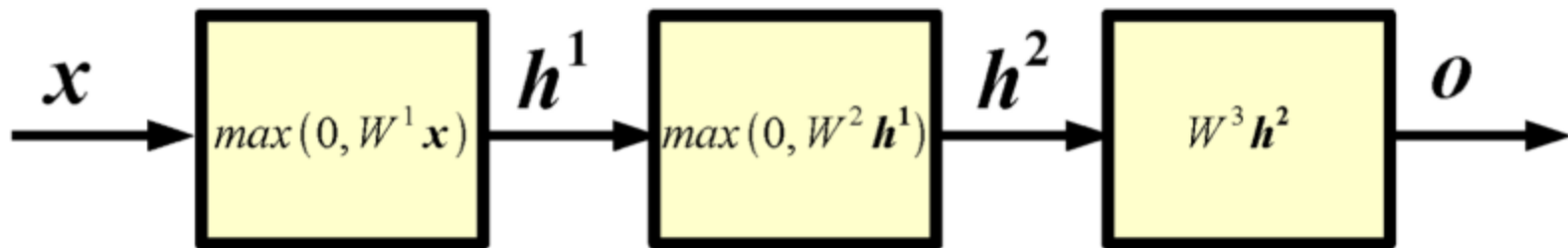
Question: What does a hidden unit do?

Question: How many layers? How many hidden units?

Question: How do I set the weight matrices?

Multiple Layer neural network

Neural Networks: example



x input

h^1 1-st layer hidden units

h^2 2-nd layer hidden units

o output

Example of a 2 hidden layer neural network (or 4 layer network, counting also input and output).



Supervised Learning

$\{(\mathbf{x}^i, y^i), i=1 \dots P\}$ training dataset

$\mathbf{x}^i$ i-th input training example

y^i i-th target label

P number of training examples



Goal: predict the target label of unseen inputs.

Examples

Supervised Learning Example

Classification



"dog"

Denoising



OCR



"2 3 4 5"

classification

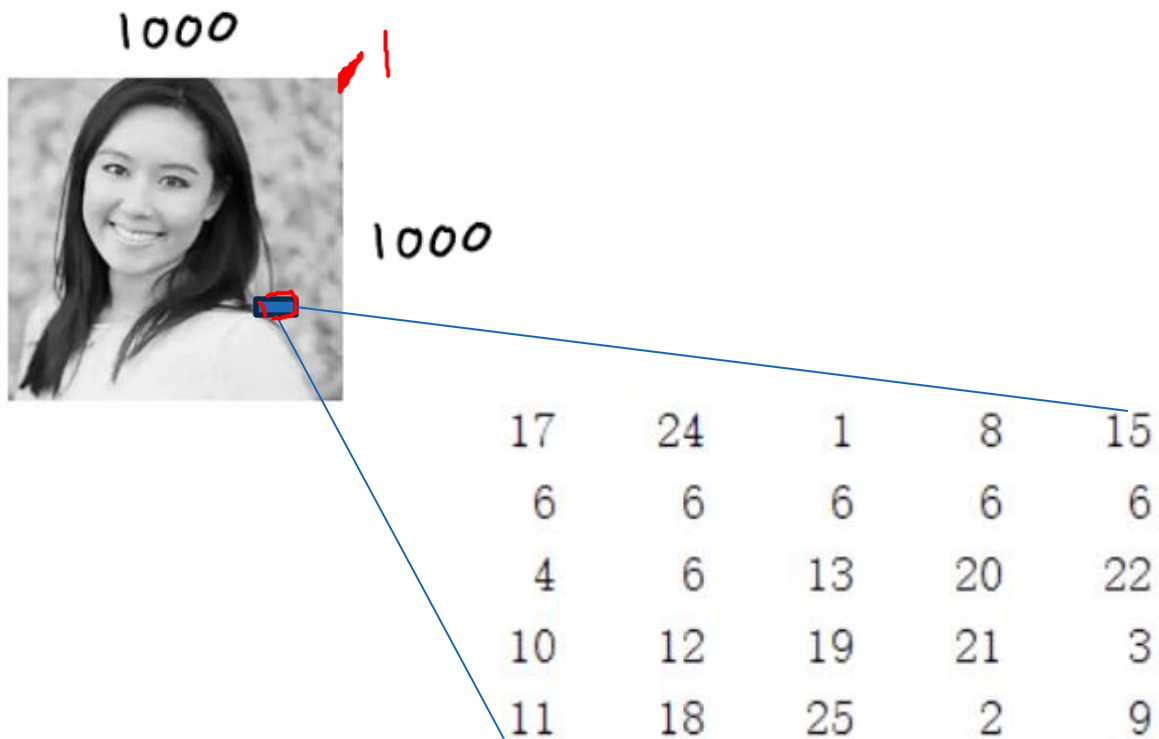
regression

structured prediction

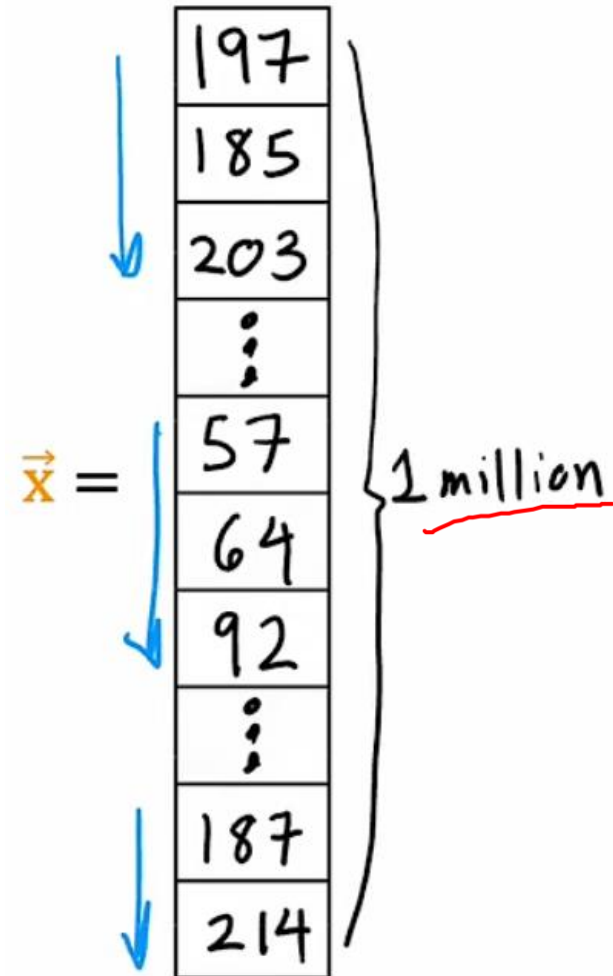
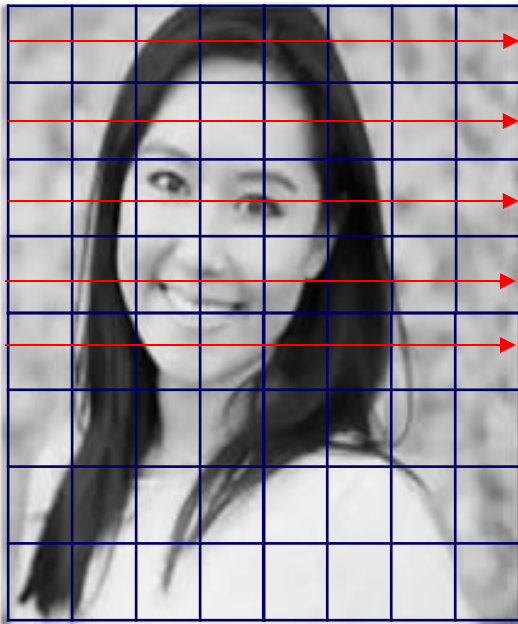
3

Ranzato 

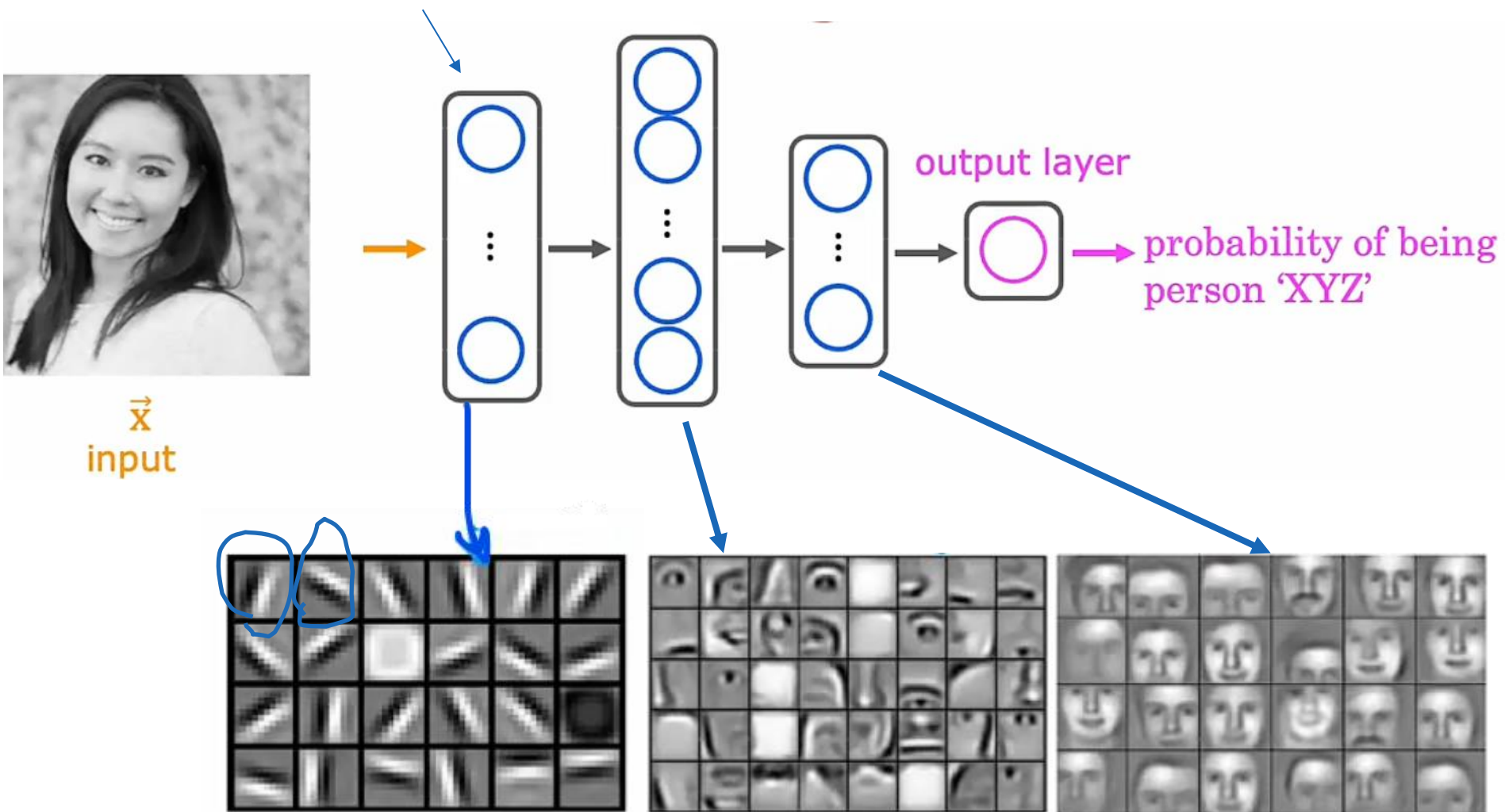
Face recognition



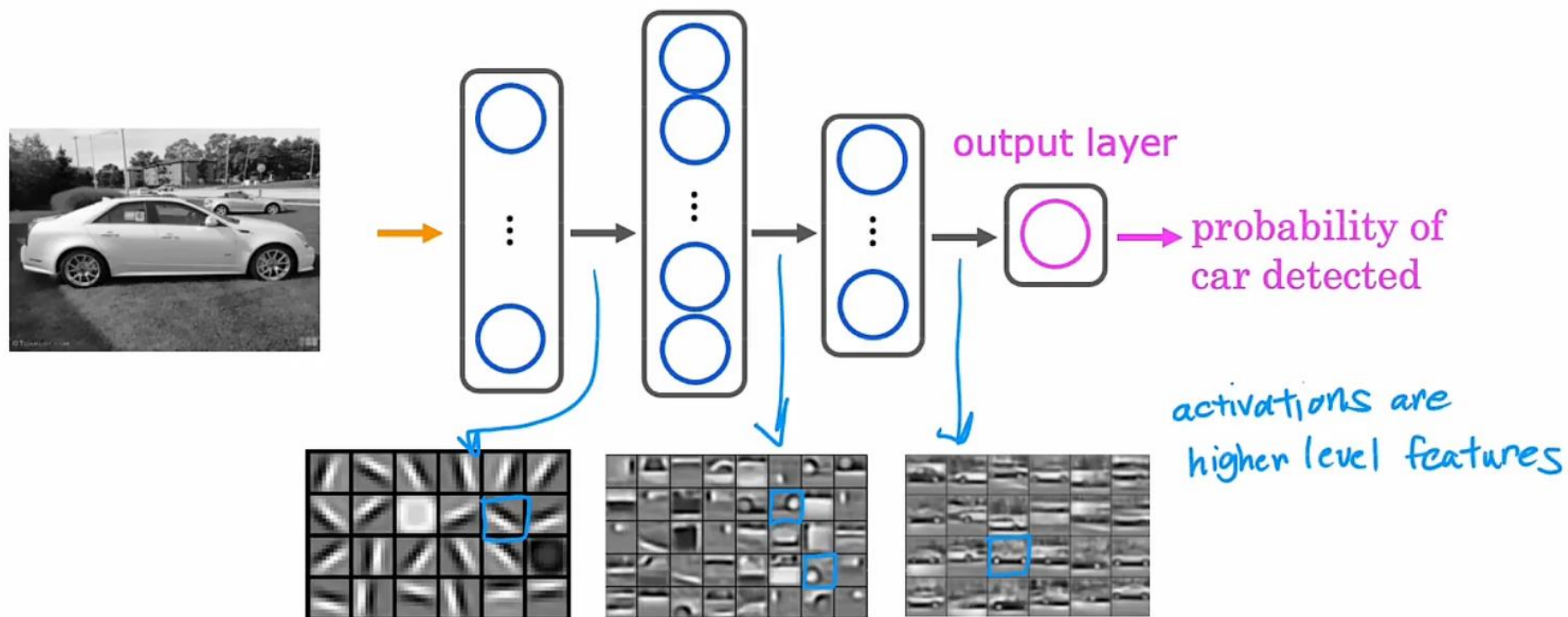
Face recognition



Face recognition

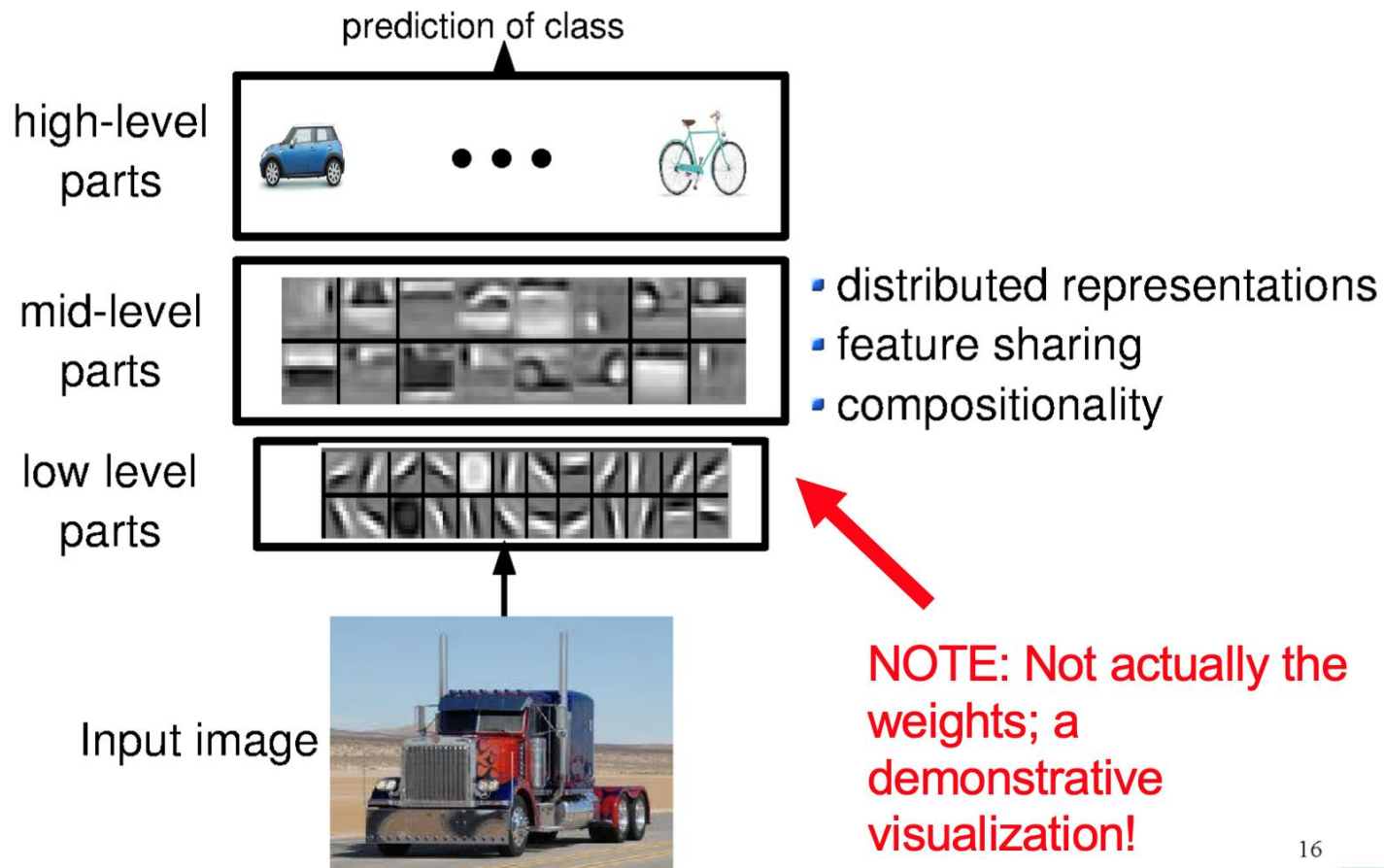


Car classification



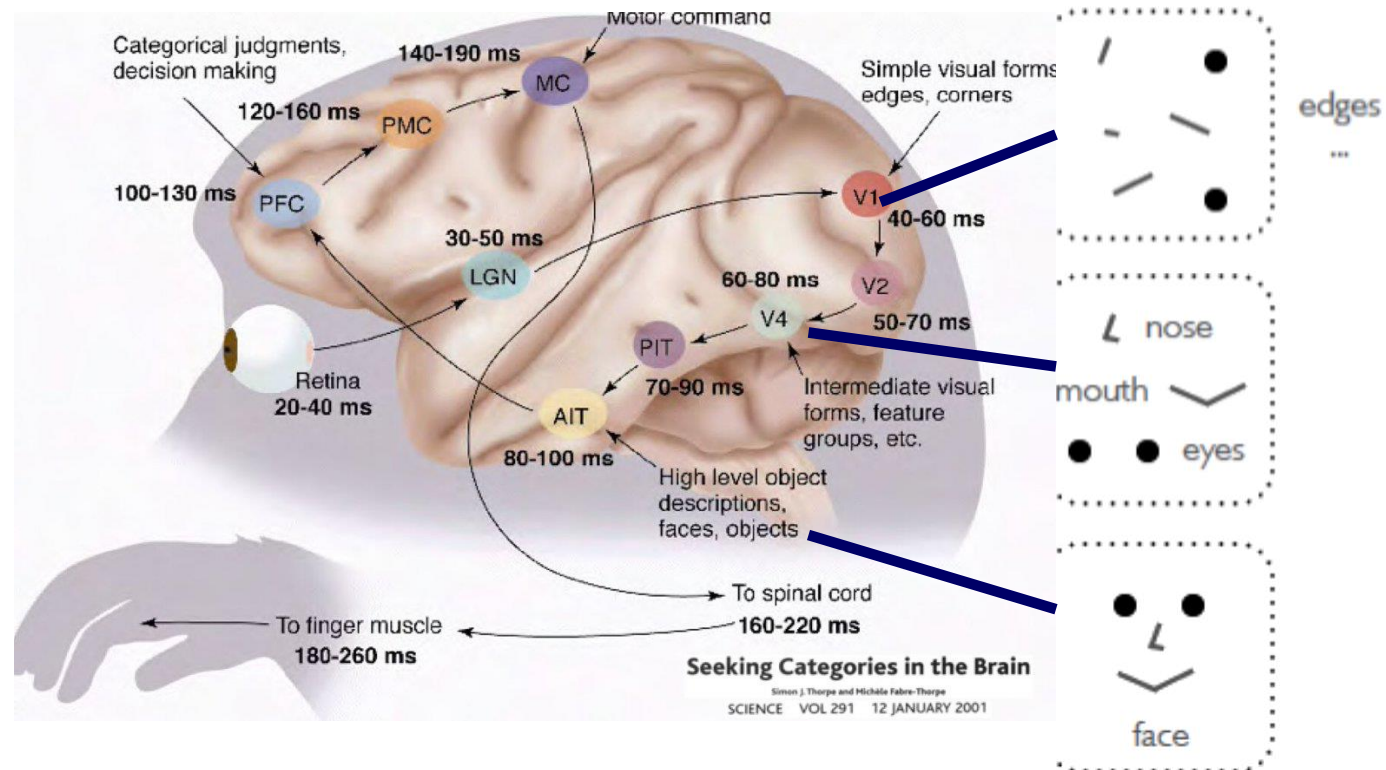
source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations

Interpretation

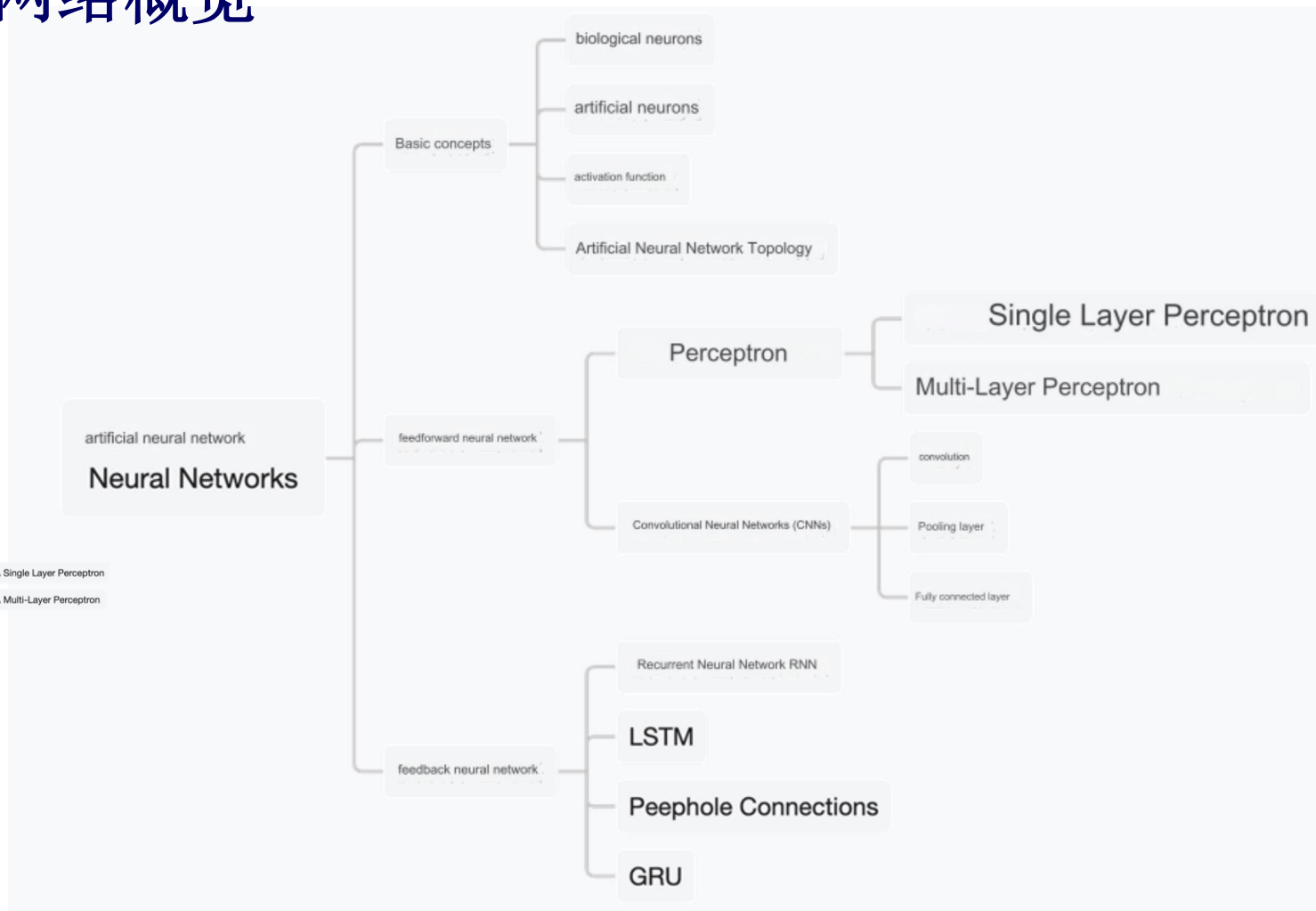


Lee et al. "Convolutional DBN's ..." ICML 2009

Biological inspiration: Human Neural Systems



❖ 人工神经网络概览





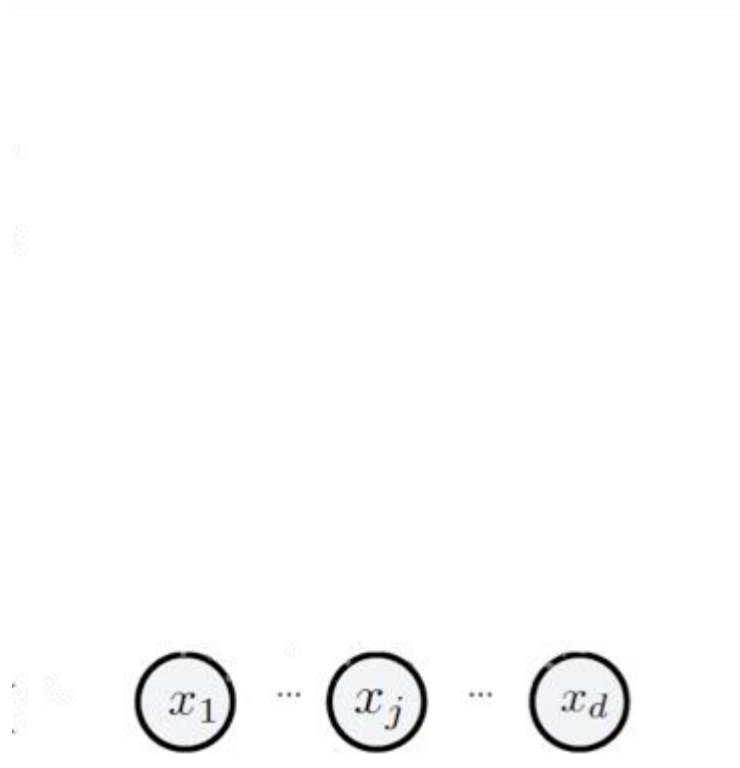
Multilayer Neural Networks

❖ ***Forward Computation***



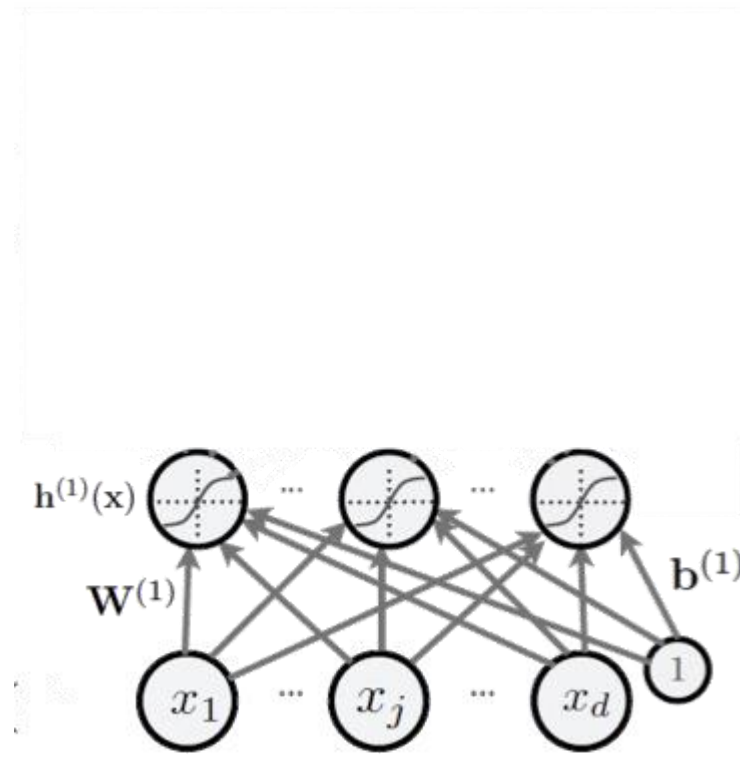
Multilayer Neural Network

❖ Forward



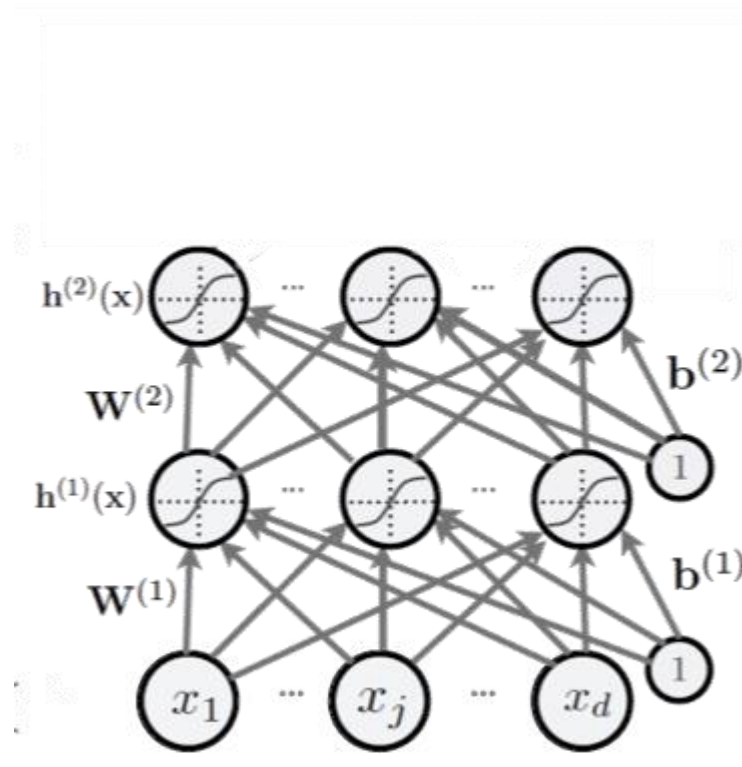
Multilayer Neural Network

❖ Forward



Multilayer Neural Network

❖ Forward



Multilayer Neural Network

Recursion
递归

- Could have L hidden layers:
 - Layer pre-activation for $k > 0$:

$$\mathbf{a}^{(k)}(\mathbf{x}) = \mathbf{b}^{(k)} + \mathbf{W}^{(k)}\mathbf{h}^{(k-1)}(\mathbf{x})$$

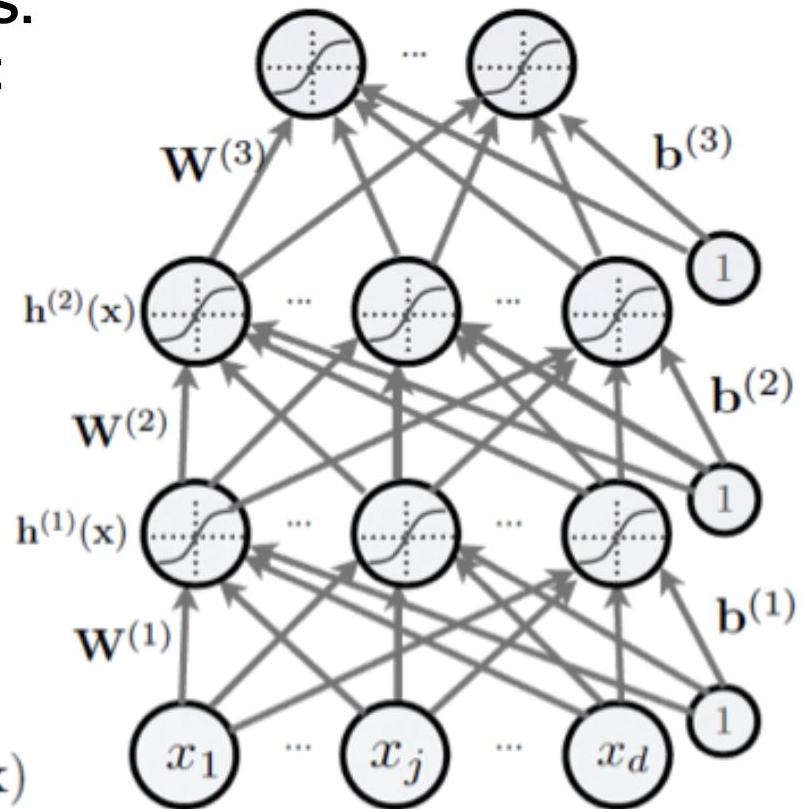
$$(\mathbf{h}^{(0)}(\mathbf{x}) = \mathbf{x})$$

- Hidden layer activation
(k from 1 to L):

$$\mathbf{h}^{(k)}(\mathbf{x}) = \mathbf{g}(\mathbf{a}^{(k)}(\mathbf{x}))$$

- Output layer activation
($k=L+1$):

$$\mathbf{h}^{(L+1)}(\mathbf{x}) = \mathbf{o}(\mathbf{a}^{(L+1)}(\mathbf{x})) = \mathbf{f}(\mathbf{x})$$



Multilayer Neural Network

- ❖ **Multilayer neural network models a function $f(x, \{w_i, b_i\})$ by multiple layers of neurons.**
- ❖ ***The goal of using neural network is to model the data by solving the weight parameters, $\{w_i, b_i\}$, of the neural network through an optimization process which is defined via the backpropagation.***

❖ 多层神经网络通过多层神经元对函数 $f(x, \{w_i, b_i\})$ 进行建模。

❖ 使用神经网络的目的是通过反向传播定义的优化过程求解神经网络的权重参数 $\{w_i, b_i\}$ 来对数据进行建模。



Machine Learning?

- In our case, this means:
 - Need to find a way to identify good values for our network's weights
 - Need to do this by relying on training examples (past experiences)
 - Need to do this by employing some sort of performance measure
- Loss function:
mathematical minimization!





Empirical risk minimization

- Framework to design learning algorithms

$$\arg \min_{\boldsymbol{\theta}} \frac{1}{T} \sum_t l(f(\mathbf{x}^{(t)}; \boldsymbol{\theta}), y^{(t)}) + \lambda \Omega(\boldsymbol{\theta})$$

- $l(f(\mathbf{x}^{(t)}; \boldsymbol{\theta}), y^{(t)})$ = loss function
- $\Omega(\boldsymbol{\theta})$ = regularizer (penalizes certain values of $\boldsymbol{\theta}$)

- Learning is cast as mathematical optimization
 - Ideally, we optimize classification errors, but it's not smooth
 - Loss function is a surrogate for what we truly should optimize (e.g. upper bound)





Classification error is non-smooth and non-convex

$$\hat{f}^{\text{erm}} = \operatorname{argmin}_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \mathbb{I}(Y_i \neq f(X_i)).$$

The following is an example of convex surrogates of loss functions.

- **Hinge loss:** $\ell(y) = \max(0, 1 - t \cdot y)$



Recipe for design a Machine Learning system

1. Given training data:

$$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N$$

2. Choose each of these:

– Decision function

$$\hat{\mathbf{y}} = f_{\boldsymbol{\theta}}(\mathbf{x}_i)$$

– Loss function

$$\ell(\hat{\mathbf{y}}, \mathbf{y}_i) \in \mathbb{R}$$

3. Define Goal state:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^N \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

4. Train with BP in SGD:
(take small steps opposite the gradient)

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

Loss function for classification

- Neural network estimates $f(\mathbf{x})_c = p(y = c|\mathbf{x})$
 - We could maximize the probabilities of $y^{(t)}$ given $\mathbf{x}^{(t)}$ in the training set

- To frame as minimization, we minimize the negative log-likelihood

$$l(\mathbf{f}(\mathbf{x}), y) = - \sum_c 1_{(y=c)} \log f(\mathbf{x})_c = - \log f(\mathbf{x})_y$$

Natural log (ln)



- We take the log to simplify for numerical stability and math simplicity
- Sometimes referred to as cross-entropy

Regularization

- L2-regularization $\Omega(\theta) = \sum_k \sum_i \sum_j \left(W_{i,j}^{(k)}\right)^2 = \sum_k \|\mathbf{W}^{(k)}\|_F^2$
 - Only applied on weights, not on biases (weight decay)
 - Can be interpreted as having a Gaussian prior on weights
- L1-regularization

$$\Omega(\theta) = \sum_k \sum_i \sum_j |W_{i,j}^{(k)}|$$

- Again, only applied on weights
- Unlike L2, L1 will push certain weights to be exactly 0
- Can be interpreted as having a Laplacian prior on weights

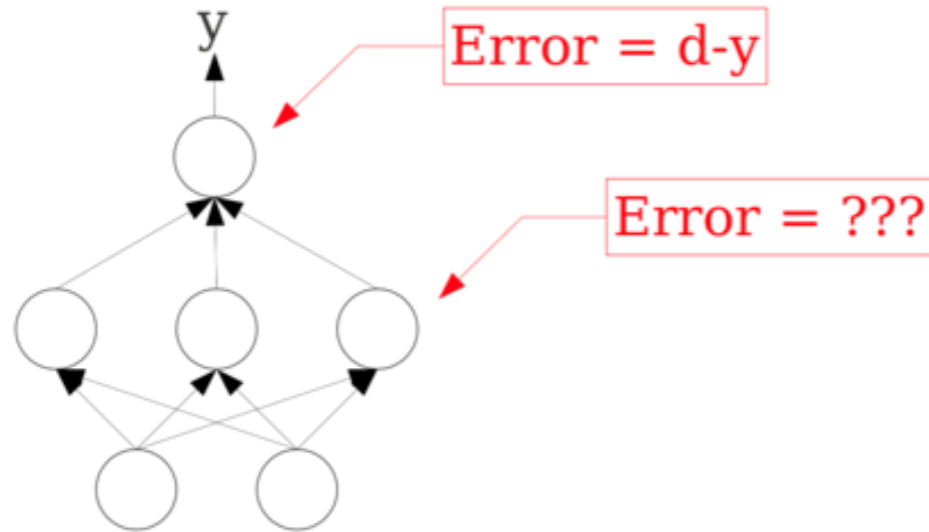
Learning Algorithm

- Algorithm that performs updates after input each data:

$$\theta \quad (\theta \equiv \{ \mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \dots, \mathbf{W}^{(L+1)}, \mathbf{b}^{(L+1)} \})$$

- Initialize $(\mathbf{x}^{(t)}, y^{(t)})$
 - For N iterations: $\theta \leftarrow \theta + \alpha \Delta$
 - iterate through each training example
- } Training epoch
=
Iteration over all
examples

How Do We Train A Multi-Layer Network?



$$\arg \min_{\theta} \frac{1}{T} \sum_t l(f(\mathbf{x}^{(t)}; \theta), y^{(t)}) + \lambda \Omega(\theta)$$

Gradient descent

- Based on observation that
 - If multi-variate function $f(\boldsymbol{\theta})$ defined and differentiable in neighborhood of a point $\boldsymbol{\theta}_0$
 - Then $f(\boldsymbol{\theta})$ decreases fastest if one goes from $\boldsymbol{\theta}_0$ in the direction of the negative gradient of $f(\boldsymbol{\theta})$ at $\boldsymbol{\theta}_0$
 - The gradient of $f(\boldsymbol{\theta})$ in $\boldsymbol{\theta}_0$ is denoted $\nabla f(\boldsymbol{\theta}_0)$
 - It follows that, for small enough γ

$$\boldsymbol{\theta}_1 = \boldsymbol{\theta}_0 - \gamma \cdot \nabla f(\boldsymbol{\theta}_0)$$

we have $f(\boldsymbol{\theta}_1) < f(\boldsymbol{\theta}_0)$

- If executing $\boldsymbol{\theta}_{n+1} = \boldsymbol{\theta}_n - \gamma \cdot \nabla f(\boldsymbol{\theta}_n)$

we obtain $f(\boldsymbol{\theta}_n) < f(\boldsymbol{\theta}_{n-1}) < \dots < f(\boldsymbol{\theta}_2) < f(\boldsymbol{\theta}_1) < f(\boldsymbol{\theta}_0)$

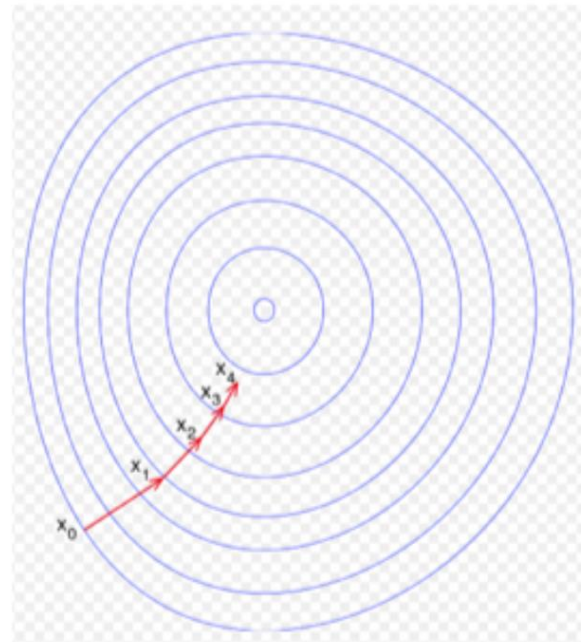
Note: here the notation $f(\cdot)$ is a general objective function to be minimised.

Gradient descent

- With properly chosen γ in each iteration
 - Convergence to local minimum!
 - γ can be adapted between iterations

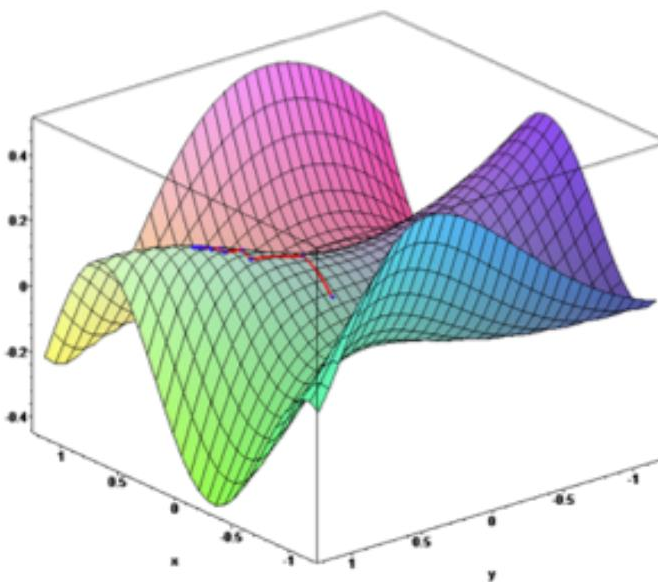
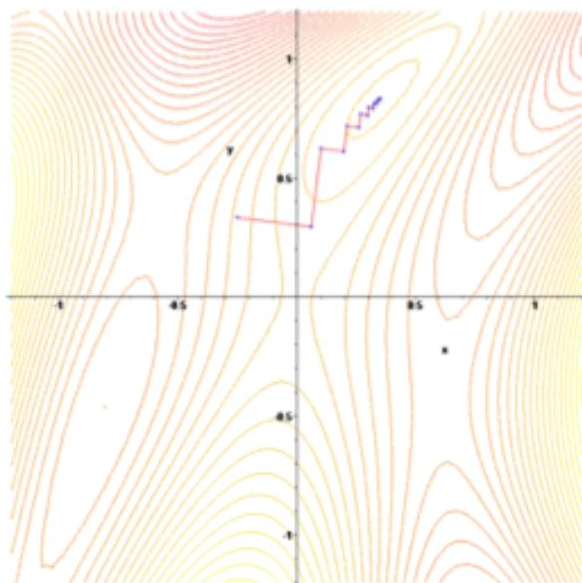
or

- We can iterate over γ inside individual iterations
 - Example:
line-search

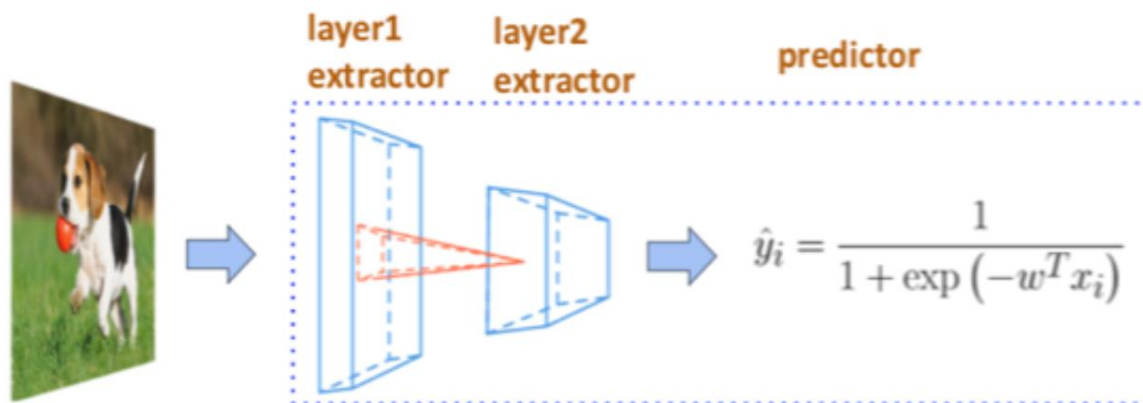


Gradient descent

- Works in arbitrary dimensions



Model Training Overview



Objective

$$L(w) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \lambda \|w\|^2$$

Training

$$w \leftarrow w - \eta \nabla_w L(w)$$

Learning Algorithm

- Initialize θ ($\theta \equiv \{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \dots, \mathbf{W}^{(L+1)}, \mathbf{b}^{(L+1)}\}$)
 - For N iterations:
 - For each training example $(\mathbf{x}^{(t)}, y^{(t)})$
 - $-\Delta = -\nabla_{\theta} l(f(\mathbf{x}^{(t)}; \theta), y^{(t)}) - \lambda \nabla_{\theta} \Omega(\theta)$
 - $\theta \leftarrow \theta + \alpha \Delta$
- Training epoch
=
Iteration over all
examples
- To apply this algorithm to neural network training, we need
 - The loss function $l(\mathbf{f}(\mathbf{x}^{(t)}; \theta), y^{(t)})$
 - Procedure to compute parameter gradients $\nabla_{\theta} l(\mathbf{f}(\mathbf{x}^{(t)}; \theta), y^{(t)})$
 - The regularizer $\Omega(\theta)$ (and the gradient $\nabla_{\theta} \Omega(\theta)$)
 - Initialization method



Training deep neural networks

- Outlook:
 - Computing gradients is hard (many parameters)
→ Back-propagation!
- Network represents a chain of function calls (one per layer):

$$\begin{aligned} f(\mathbf{x}) &= f_{L+1} \circ f_L \circ \dots \circ f_1(\mathbf{x}) \\ &= f_{L+1}(f_L(\dots f_1(\mathbf{x}))) \end{aligned}$$

- Gradients: Chain rule can be applied!

$$\frac{\partial f(\mathbf{x})}{\partial \boldsymbol{\theta}} = \frac{\partial f_{L+1}}{\partial \mathbf{f}_L} \cdot \frac{\partial f_L}{\partial \mathbf{f}_{L-1}} \cdot \dots \cdot \frac{\partial f_1}{\partial \boldsymbol{\theta}}$$



How to compute gradient?

Symbolic Differentiation

- Input formulae is a symbolic expression tree (computation graph).
- Implement differentiation rules, e.g., sum rule, product rule, chain rule

$$\frac{d(f + g)}{dx} = \frac{df}{dx} + \frac{dg}{dx} \quad \frac{d(fg)}{dx} = \frac{df}{dx}g + f\frac{dg}{dx} \quad \frac{d(h(x))}{dx} = \frac{df(g(x))}{dx} \cdot \frac{dg(x)}{dx}$$

- ✗ For complicated functions, the resultant expression can be exponentially large.
- ✗ Wasteful to keep around intermediate symbolic expressions if we only need a numeric value of the gradient in the end
- ✗ Prone to error

How to compute gradient?

Numerical Differentiation

- We can approximate the gradient using

$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}$$

$$f(W, x) = W \cdot x$$
$$[-0.8 \quad 0.3] \cdot \begin{bmatrix} 0.5 \\ -0.2 \end{bmatrix}$$

How to compute gradient?

Numerical Differentiation

- We can approximate the gradient using

$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}$$

$$f(W, x) = W \cdot x \\ [-0.8 \quad 0.3] \cdot \begin{bmatrix} 0.5 \\ -0.2 \end{bmatrix}$$

$$f(W, x) = W \cdot x \\ [-0.8 + \epsilon \quad 0.3] \cdot \begin{bmatrix} 0.5 \\ -0.2 \end{bmatrix}$$

How to compute gradient?

Numerical Differentiation

- We can approximate the gradient using

$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}$$

- Reduce the truncation error by using center difference

$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x} - h\mathbf{e}_i)}{2h}$$

✗ Bad: rounding error, and slow to compute

✓ A powerful tool to check the correctness of implementation, usually use $h = 1e-6$.

End of Lecture 3

Can you understand artificial neural network and apply it to special applications?

LIUXIABI@BIT.EDU.CN

